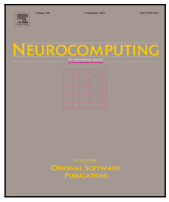
Neurocomputing 610 (2024) 128511

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom



Review of neural network model acceleration techniques based on FPGA platforms

Fang Liu [a,b,*], Heyuan Li [c], Wei Hu [c], Yanxiang He [a]

[a] School of Computer Science, Wuhan University, Wuhan, China
[b] Wuhan Vocational College of Software and Engineering, Wuhan, China
[c] College of Computer Science, Wuhan University of Science and Technology, Wuhan, China

ARTICLE INFO

Keywords:
Neural network model
FPGA
Algorithm hardware collaboration
Acceleration and optimization

ABSTRACT

Neural network models, celebrated for their outstanding scalability and computational capabilities, have demonstrated remarkable performance across various fields such as vision, language, and multimodality. The rapid advancements in neural networks, fueled by the deep development of Internet technology and the increasing demand for intelligent edge devices, introduce new challenges, including significant model parameter sizes and increased storage pressures. In this context, Field-Programmable Gate Arrays (FPGA) emerge as a preferred platform for accelerating neural network models, thanks to their exceptional performance, energy efficiency, and the flexibility and scalability of the system. Building FPGA-based neural network systems necessitates bridging significant differences in objectives, methods, and design spaces between model design and hardware design. This review article adopts a comprehensive analytical framework to thoroughly explore multidimensional technological implementation strategies, encompassing optimizations at the algorithmic and hardware levels, as well as compiler optimization techniques. It focuses on methods for collaborative optimization between algorithms and hardware, identifies challenges in the collaborative design process, and proposes corresponding implementation strategies and key steps. Addressing various technological dimensions, the article provides in-depth technical analysis and discussion, aiming to offer valuable insights for research on optimizing and accelerating neural network models in edge computing environments.

1. Introduction

1.1. Research background and significance

The rapid advancement of deep learning technologies has led to significant achievements across multiple domains, including image recognition, speech recognition, and natural language processing [1–3]. To tackle increasingly complex problems and achieve higher accuracy, the structural complexity of models has escalated, leading to a dramatic increase in the number of parameters. While this complexity enhancement has considerably improved performance, it has also substantially raised the demands for computational power and storage space, leading to increased time and energy consumption during model training and inference. Despite continuous advancements in hardware technology and the significant enhancement of GPU parallel processing capabilities, the pace of hardware development still lags behind the rapid growth of deep learning model complexity. The slowdown of Moore's Law signals that relying solely on hardware performance improvements to compensate for algorithmic complexity increases is facing a bottleneck. Particularly in terms of power efficiency, the energy efficiency issue of high-performance computing has become a critical factor hindering the sustainable development of deep learning technologies, limiting their widespread deployment in energy-sensitive applications.

Confronted with the escalating contradiction between model complexity and computational capability, there is a dual necessity: on one hand, to continuously enhance model scale and complexity in pursuit of better performance; on the other hand, considerations of computational resource limitations, energy consumption, and deployment costs strictly constrain model scaling. This contradiction is especially pronounced in edge computing scenarios, which demand higher requirements for real-time performance and energy efficiency of models. Thus, exploring new computational platforms and architectures, such as Field-Programmable Gate Arrays (FPGAs) known for their high flexibility and efficiency, has become a critical direction in deep learning research. FPGAs effectively accelerate model operational efficiency in terms of real-time inference and energy efficiency ratio. The pursuit

* Corresponding author at: School of Computer Science, Wuhan University, Wuhan, China.
E-mail address: liufangfang@whu.edu.cn (F. Liu).

https://doi.org/10.1016/j.neucom.2024.128511
Received 13 March 2024; Received in revised form 7 August 2024; Accepted 27 August 2024
Available online 31 August 2024

of computational efficiency improvements through hardware acceleration and hardware-software co-design to mitigate the conflict between model complexity growth and computational capacity enhancement holds significant academic value and broad application prospects.

The deployment of Neural Network Model onto FPGA encompasses a complex interaction between model design and hardware design, presenting a multifaceted and challenging construction of FPGA-based systems. The significant disparities in objectives and methodologies between model design and hardware design constitute a considerable challenge. Model design aims to efficiently map and implement neural network models for effective inference or training operations on FPGA platforms, which includes maximizing the use of FPGA's parallel computing capabilities and optimizing computational efficiency. Conversely, the goal of hardware design is to create a reliable and high-performance system to support model operation, necessitating optimization around timing, resource allocation, and system stability. These differences require designers to find a balance between model design and hardware implementation.

Although existing research has explored challenges faced by FPGA-based neural model accelerators, such as the "memory wall" issue and acceleration techniques for specific task models, there is a lack of comprehensive discussion on the disparities between model and hardware design. Many studies focus on model acceleration in specific domains, missing a holistic analysis on the scalability and design automation for implementing different types of network models on FPGAs. Hence, research into FPGA-based acceleration techniques for Neural Network Model not only holds significant academic value but also has extensive practical application prospects.

A deep understanding of the differences between model design and hardware design, and exploring effective methods to overcome these differences, are crucial for promoting the widespread deployment of deep learning technologies in edge computing and real-time application scenarios. Future research needs to focus not only on technological advancements and optimizations but also on deeper discussions and innovations in design automation and system scalability. Through such research, we can take significant steps towards realizing more efficient and intelligent FPGA-accelerated deep learning systems.

1.2. Research scope and objectives

The deployment of Neural Network Model onto FPGA involves a delicate balance between model design and hardware design. Model design is aimed at achieving efficient neural network model mapping, while hardware design focuses on realizing these models on FPGA platforms, addressing critical technical issues such as resource allocation and timing design. Although closely related, the objectives and methodologies of these two aspects significantly differ, necessitating an effective collaborative pathway between model design and hardware implementation.

This study aims to provide a comprehensive summary and in-depth analysis of FPGA-based deep learning model acceleration techniques. It categorizes and analyzes the different steps and design objectives from network models to the construction of a runnable system. While existing reviews and research have detailed the technical and design methods involved, they often overlook a thorough analysis of the objectives of these design solutions, lacking a holistic description and division of the construction of FPGA-based neural network systems and the connections between different dimensional design solutions. By analyzing the differences and connections between deep learning model design and FPGA hardware design, this research explores how to establish an effective collaboration mechanism between the two, aiming for efficient model mapping and hardware realization. The construction process of FPGA-based neural network systems is meticulously categorized, providing deep insights from the perspectives of different steps and design objectives, revealing the technical requirements and design choices at each step.

From the perspective of the overall construction process, this paper offers a comprehensive summary and in-depth analysis of network model design solutions based on FPGAs. It systematically clarifies the differences between deep learning network model design and FPGA hardware design and identifies key integration points, providing a theoretical foundation and practical guide for collaborative design. This paper establishes a clear classification framework, dividing different design solutions in the construction process of FPGA-based neural network systems according to technical layers, and elaborately discussing their technical characteristics and application scenarios. Finally, by analyzing and summarizing existing automated design solutions, this research explores how to further enhance the automation level and design efficiency of FPGA-based deep learning model acceleration technology, especially in terms of direct model-to-hardware mapping and optimization (see Fig. 1).

1.3. Problems and challenges

The development of FPGA-based acceleration techniques for Neural Network Model is confronted with multifaceted challenges, including limitations on resources, design complexity, optimization of parallel computing, power consumption management, design automation and tool support, as well as the demands for scalability and flexibility.

1. Resource Limitations: Primarily, this refers to the constraints on model size and resources, along with storage and communication bandwidth limitations. FPGA chips' resources encompass logic units, lookup tables (LUTs), registers, block RAMs (BRAMs), and digital signal processor (DSP) units, among others. The requirement for substantial computational power and storage capacity by large-scale Neural Network Model contrasts sharply with the inherent resource limitations of FPGAs. Model compression techniques (e.g., pruning, parameter sharing) and quantization algorithms (converting 32-bit floating-point parameters to fixed-point representations with lower bit-widths) are extensively applied to reduce model size and computational demands.

2. Storage and Communication Bandwidth Limitations: The constraints on FPGA's storage capacity and communication bandwidth impact data processing speed and model scalability. Efficient dataflow design and storage management techniques, such as loop unrolling and data reuse, are utilized to optimize storage access patterns and minimize dependency on external storage, thereby enhancing storage and communication efficiency.

3. Algorithm-Hardware Misalignment: The parallelism and timing characteristics of FPGAs do not match well with the matrix computations and convolution operations of deep learning algorithms. Effective mapping of algorithms requires refined management of FPGA's hardware resources, including the configuration of logic units, allocation of storage resources, and design of data paths. This process necessitates a deep understanding of FPGA's hardware characteristics and targeted algorithm optimization and hardware design.

4. Computational Architecture Design: Designing efficient parallel computing architectures requires consideration of data dependencies, computational intensity, and memory access patterns. Implementing parallel acceleration of Neural Network Model on FPGAs involves optimizing computation latency and throughput through techniques like pipelining, data parallelism, and model parallelism, including the design of efficient data flow pipelines and memory access strategies, as well as innovation in parallel algorithms.

5. Data Flow Design and Storage Management: To fully leverage FPGA's parallel processing capabilities, efficient data flow pipelines must be designed to ensure effective data transfer between different computing units. Additionally, reasonable storage management strategies can minimize data access latency and storage overhead, including optimizing memory layout, implementing locality optimizations, and utilizing on-chip storage resources.

6. Power Consumption and Efficiency: The balance between power consumption and efficiency is crucial. Power management in FPGAs

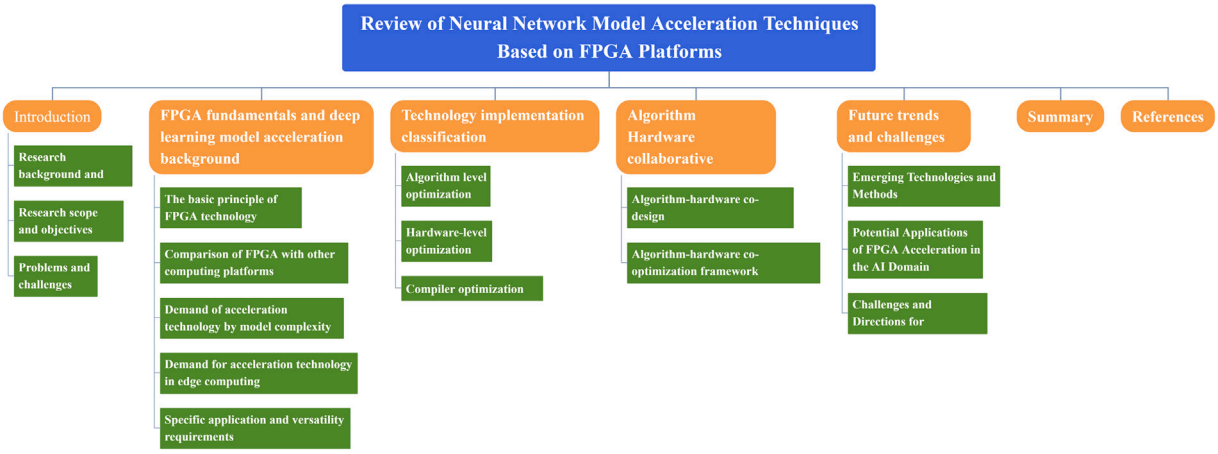


Fig. 1. Framework of this paper.

is key to optimizing the energy efficiency of computing and storage units. Considering the energy efficiency of algorithms and low-power hardware design techniques is vital when designing deep learning accelerators. Effective power reduction can be achieved by adjusting computational precision, employing dynamic voltage and frequency scaling (DVFS) techniques, and optimizing the scheduling and allocation of computational tasks.

7. Lack of Automated Design Tools: Currently, the transformation of Neural Network Model to FPGA implementations largely relies on manual design, which is time-consuming and prone to errors. Enhancing the level of design automation, such as developing tools and frameworks capable of automatically performing model compression, quantization, mapping, and optimization, is key to improving development efficiency and accelerating the deployment of Neural Network Model.

The advancement of FPGA-based deep learning model acceleration technology faces several technical challenges that necessitate a deep understanding of deep learning algorithms and FPGA hardware characteristics, and innovation and optimization in model compression, algorithm optimization, hardware design, and the development of automation tools.

2. FPGA fundamentals and deep learning model acceleration background

2.1. The basic principle of FPGA technology

FPGA are highly flexible digital integrated circuits, core to advancing digital design and application adaptability. An FPGA's architecture consists of thousands of Configurable Logic Blocks (CLBs), programmable interconnects, and Input/Output Blocks (IOBs). CLBs, housing fundamental logic units such as flip-flops, Look-Up Tables (LUTs), and Boolean logic gates, facilitate the implementation of complex logic and storage functions through programming. The programmable interconnect resources enable flexible linking of these logic blocks to form intricate digital circuits. FPGAs often incorporate specialized hardware blocks, such as Digital Signal Processing (DSP) blocks and RAM blocks, to optimize specific computational tasks.

The operational principle of FPGAs revolves around loading a predefined configuration file (bitstream) into the chip. This bitstream meticulously dictates the configuration details of logic blocks and interconnects. Once configured, FPGAs process input data based on these preset parameters, yielding results according to the designed logic circuits. This capacity for handling complex algorithms and managing high-speed data streams underscores FPGAs' exceptional performance, particularly in digital tasks demanding high performance and parallel processing capabilities. FPGAs' ability to rapidly switch between applications without hardware replacement facilitates adaptation to new functional requirements.

The advantages of FPGAs lie in their flexibility, superior parallel processing performance, and the acceleration of prototype development. Users can reconfigure FPGAs to execute varied hardware functions tailored to specific application needs. In tasks requiring extensive parallel computations, FPGAs offer superior performance compared to traditional microprocessors. The rapid prototype development feature positions FPGAs as an ideal platform for new hardware design and testing new concepts, thereby accelerating the development cycle from concept to product. Consequently, FPGAs play a pivotal role in several domains, including communications, image processing, and embedded systems (see Fig. 2).

2.2. Comparison of FPGA with other computing platforms

Since the mid-20th century, the advent of computer technology, particularly the development of processor chips, has significantly transformed human life. The Central Processing Unit (CPU), the cornerstone of information technology, employs a general-purpose processor chip design philosophy, powering deep applications across various industries. Designed to meet general computing needs, CPUs enable diverse application functionalities through software reprogramming without hardware modifications, significantly enhancing productivity while reducing labor and capital investments. Advances in semiconductor integrated circuit technology, including transistor miniaturization and computational speed enhancement, have notably improved chip performance. Gordon Moore, one of Intel's founders, proposed Moore's Law, which posits that the number of transistors on integrated circuits doubles approximately every two years, guiding semiconductor industry development for over half a century. During this period, increases in CPU transistor counts and clock frequencies, coupled with continuous innovation and optimization in computer architecture, including microarchitecture, cache technology, and parallel computing, have ensured sustained performance improvements while maintaining generality.

However, in the last decade, as semiconductor device sizes approach physical limits, Moore's Law and Dennard scaling have encountered bottlenecks, leading to a slowdown in general processor performance improvements. This period coincided with the rapid development of big data and deep learning technologies, triggering an explosive demand for computational power and accelerating the shift towards specialized computing. Compared to general-purpose CPUs, specialized chips, designed for specific applications, offer significant performance and efficiency improvements in their respective fields, despite their narrower application range and weaker generality. Domain-Specific

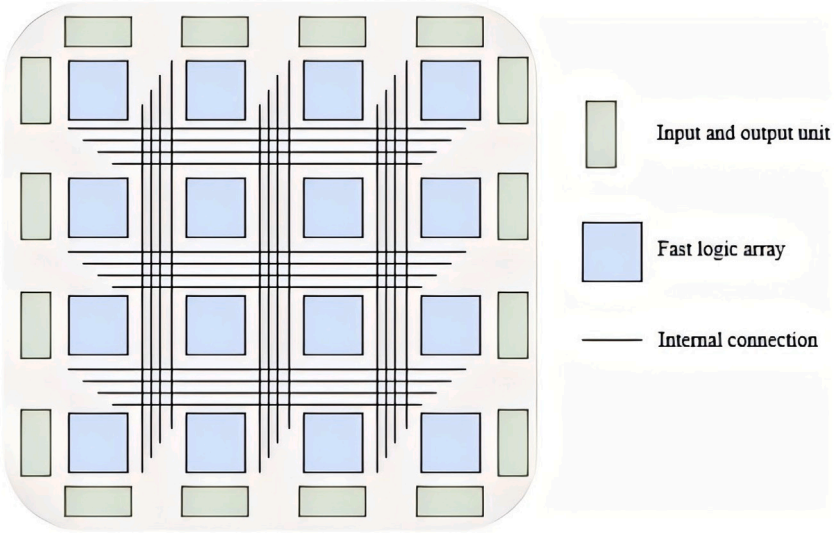


Fig. 2. FPGA chip architecture.

Architectures (DSAs), focusing on hardware accelerators for applications with similar characteristics, have emerged as a research focus in recent years, achieving significant gains in performance and energy efficiency. DSA designs aim not only for single algorithm acceleration but are based on a deep understanding of specific domains, proposing specialized architectures to meet unique domain requirements.

In the realm of domain-specific computing, the Graphical Processing Unit (GPU) has emerged as a mature platform, designed for large-scale data parallel processing with an extensive array of parallel computation units. This architectural design enables tasks, which face performance bottlenecks on Central Processing Units (CPUs), to achieve significant performance improvements on GPUs. Moreover, Application-Specific Integrated Circuits (ASICs), through comprehensive customization of hardware circuits, offer unparalleled computation speed and low power consumption for specific domains such as deep learning accelerators. However, their high development costs and lack of programmability limit their flexibility.

FPGA, with their unique blend of flexibility and performance, serve as an intermediary between GPUs and ASICs. Unlike GPUs, which are geared towards large-scale data parallel processing with numerous parallel computation units, FPGAs can be reconfigured at the hardware level. This capability allows for optimization towards specific algorithms and models, especially in applications requiring low latency and high throughput, where FPGAs demonstrate significant performance advantages. In contrast to ASICs, which can deliver extreme computation speeds and low power consumption in certain domains like deep learning acceleration, their high development costs and non-programmability restrict their adaptability. FPGAs, with their reconfigurable nature, offer greater flexibility, accommodating iterative updates to algorithms and models, thus reducing development cycles and costs.

The rise of hardware open-source trends, exemplified by the development of technologies like RISC-V and Chisel, has introduced new possibilities for the flexibility and agile development of ASICs. This movement also presents new opportunities for the application development on FPGAs. In the fields of communications and artificial intelligence, FPGA-based accelerators continue to emerge, showcasing their broad application potential in processing complex algorithms and meeting specific application needs.

Therefore, FPGA has become an efficient and flexible computing platform in the fields of deep learning and artificial intelligence, especially in the rapidly changing technological and application requirements. It can not only provide targeted hardware acceleration, but also quickly adapt to technological evolution and changes in application requirements through its reconfigurable features, meeting the efficiency and flexibility requirements of domain specific computing (see Table 1).

2.3. Demand of acceleration technology by model complexity

With the rapid development of deep learning technology, research directions are increasingly leaning towards large models. Supported by massive data and a vast number of parameters, these models perform computational tasks through deep neural networks. This trend not only drives the application of Neural Network Model across various fields but also imposes higher demands on computational hardware.

Demand for High-Performance Computing Power: Large Neural Network Model require robust computing support to handle massive data and complex computational tasks. This includes, but is not limited to, higher computation speeds, larger storage capacities, and faster data transfer rates. Although current general-purpose processor architectures, such as CPUs and GPUs, can provide relatively high computing power, their performance enhancements are beginning to reach a bottleneck with the continuous increase in model complexity.

Support for Model Characteristics: Neural Network Model are increasingly characterized by unique features, such as specific network structures and computation patterns, to achieve higher accuracy. This necessitates that computing power not only supports traditional computational tasks but also caters to the unique characteristics of models. ASICs and FPGA-based custom processor architectures have a clear advantage in this regard, as they can be customized for specific Neural Network Model, providing more efficient computing support.

Balancing Flexibility and Generality: As Neural Network Model continue to evolve, computing platforms need to offer not only powerful computing capabilities but also sufficient flexibility to adapt to updates and optimizations of models. FPGAs demonstrate their unique advantages in this aspect, capable of adapting to different model requirements through reconfigurable hardware. However, the optimization of FPGAs for specific models often sacrifices generality, which is a major challenge currently faced in the field of deep learning acceleration.

The increasing complexity of Neural Network Model presents new challenges for computational hardware, demanding not only high-performance computing capabilities but also the ability to support specific features of models and find a balance between flexibility and generality.

2.4. Demand for acceleration technology in edge computing

Artificial neural networks simulate the structure and functioning of human brain neurons through mathematical models, enabling efficient

Table 1
Comparison of different computing platforms.

	Design objective	Parallelism	Flexibilities
CPU	Universal Computing	Low (mainly sequential processing)	High (software reprogrammable)
GPU	Massively Parallel Data Processing	High (many parallel computing units)	Medium (software programmable with some hardware constraints)
ASIC	Maximum performance and efficiency for specific tasks	Depends on the mandate	Low (hardware fixed for specific tasks)
FPGA	Maximum performance and efficiency for specific tasks	Medium to high (configurable parallelism)	Low (hardware fixed for specific tasks) maximum

recognition of input data features by providing inputs to these models. This approach mirrors biological studies of the brain's neural structure. To achieve better computational effects, artificial neural networks have evolved with increasing layers and rapidly growing parameters. This evolution has led to the emergence of Deep Neural Networks (DNNs), characterized by their deep multi-layer network structures and vast parameter sets, granting them the ability to learn higher-level and multi-dimensional features from massive datasets. Concurrently, DNNs have introduced a significant demand for computational power. With the widespread deployment and application of Internet of Things (IoT) devices and the burgeoning of edge AI, efficiently processing DNNs has become a critical issue.

Edge devices, encompassing a wide range of devices customized based on computing technology to meet application needs, include various smart sensors, wearable devices, and intelligent mobile terminals. These extensively deployed devices become vital data sources. Given their application-specific customization, these devices often face multiple constraints, including cost, size, performance, and power consumption. Especially for battery-powered edge devices, the stringent energy consumption requirements, coupled with the inherent limitations in processor performance, result in limited computational resources. Moreover, the computational tasks in edge devices frequently have real-time requirements, necessitating completion within a given timeframe, thereby imposing higher demands on computational efficiency. Deploying applications based on DNNs to edge devices necessitates consideration of these resource constraints. Model optimization to reduce network size or parameter count is essential, diminishing the demand for computational power and storage, thereby enhancing computational efficiency.

In addressing the challenges of deploying DNN applications in edge computing, Field-Programmable Gate Array (FPGA) technology demonstrates its unique value and advantages. FPGA's high reconfigurability allows for hardware-level customization to specific DNN models, offering efficient computational performance and low-power solutions. Additionally, FPGA's low latency and compact design make it highly suitable for real-time and space-constrained edge computing scenarios. Utilizing FPGAs, developers can design and deploy specialized accelerators tailored to the specific needs and characteristics of edge computing tasks, facilitating efficient processing of DNN models. Furthermore, model optimization techniques such as weight pruning and quantization can be integrated with FPGA acceleration technology to further reduce the resource consumption of DNN models on edge devices. Thus, the application of FPGAs in edge computing not only reflects their capability in enhancing computational efficiency, reducing energy consumption, and meeting real-time requirements but also showcases their potential in meeting the growing computational demands of Neural Network Model.

2.5. Specific application and versatility requirements

In deep learning applications, optimization techniques based on FPGA encounter multiple challenges, including limited resources, algorithm mapping, data flow design, compatibility with automation tools, and the need to balance specific application requirements with universality. Researchers have adopted various methods and strategies to address these challenges, with customized and generalized designs emerging as two key strategies for the efficient deployment of Neural Network Model on FPGAs. These design philosophies play an indispensable role in FPGA-based deep learning optimization. This paper will explore these two design approaches in detail.

Customized design refers to creating FPGA hardware architectures specifically tailored to particular Neural Network Model or applications. This approach optimizes hardware to meet the demands of specific algorithms, thus enabling efficient algorithm execution. The advantage of customized design lies in its ability to fully leverage the programmability of FPGAs, finely adjusting hardware structures to reduce resource consumption and increase computational speed. However, customized designs often lack versatility, as optimizations for specific applications may not be suitable for others, limiting their potential for widespread application.

In contrast, generalized design aims to develop FPGA architectures capable of adapting to a variety of Neural Network Model and algorithms. Generalized design achieves a flexible, scalable hardware architecture that can support the requirements of multiple algorithms and applications. The strength of generalized design is its high flexibility and broad applicability, enabling a single hardware platform to serve diverse deep learning tasks. However, due to the need to accommodate multiple algorithms, generalized designs may struggle to deeply optimize for any specific algorithm, potentially underperforming compared to customized designs.

In practice, customized and generalized designs are not entirely opposed but can complement and integrate with each other. For example, researchers can optimize specific Neural Network Model on a generalized FPGA architecture using programmable logic blocks, maintaining hardware universality while enhancing computational efficiency for particular models. Through this approach, FPGAs can not only meet the high-efficiency demands of specific applications but also retain the flexibility to adapt to new models and algorithms.

3. Technology implementation classification

3.1. Algorithm level optimization

In the domain of deep learning, optimization at the algorithm level is a pivotal approach to enhancing model performance, accelerating training and inference speeds, while also reducing the consumption of computational resources. This form of optimization entails meticulous

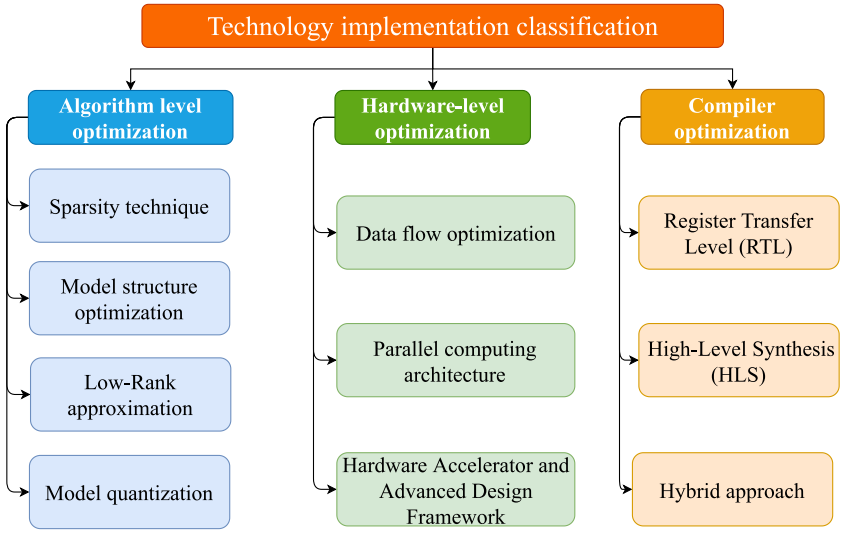


Fig. 3. Technical implementation classification framework diagram.

adjustments to neural network models and algorithms, including the optimization of data structures and the full exploitation of underlying hardware features. Through various means such as network structure optimization, parameter quantization and model compression, knowledge distillation, dynamic computation, and transfer optimization algorithms, the efficient operation of Neural Network Model is realized. This is especially critical when dealing with large-scale data and executing computations within complex, multi-layered network structures.

Beyond the optimization of models and algorithms themselves, algorithmic level optimization also encompasses the adaptation and utilization of underlying hardware characteristics. This includes specific optimizations for hardware platforms such as GPUs and FPGAs, optimizing data layouts and storage methods, leveraging hardware acceleration features, and adjusting parallel computation strategies. Such measures enable models to utilize hardware resources more efficiently (see Fig. 3).

3.1.1. *Sparsity technique*

The exceptional performance of deep neural networks stems from their capability to extract features from extensive data, positioning them as a popular choice in AI applications. The development of neural networks aimed at achieving higher accuracy has led to increasingly deep layers and larger parameter volumes, complicating their deployment on mobile and edge devices. The most crucial direction in optimizing such issues lies in balancing limited resources with model accuracy. Exceptional software design can reduce data exchange on and off the chip while significantly enhancing inference speed. Sparsification techniques are widely applied in deep learning for model compression and accelerating the inference process, primarily by reducing the number of parameters in the model to lower computational complexity and storage requirements. Sparsification can be implemented at various levels, including weight sparsification, structural sparsification, and dynamic sparsification. Model sparsification technology is a key optimization strategy in deep learning, aimed at reducing the model's computational complexity and storage demands while maintaining or even improving model performance. Depending on the classification method, sparsification techniques are categorized differently, typically into weight sparsification, structural sparsification, and dynamic sparsification.

Weight sparsification reduces model parameters by setting a portion of the weights in the neural network to zero, decreasing model complexity. This method can significantly reduce the model's storage and computational requirements without substantially affecting model accuracy. Unstructured sparsification, a form of weight sparsification, disregards the network's structure and zeroes individual weights randomly or based on certain criteria (e.g., the absolute value of weights). Another approach is structural sparsification, which sparsifies according to the structural units of the network (e.g., entire convolutional kernels or channels). Though it may sacrifice some model performance, it can reduce the irregularity of memory access and enhance computational parallelism, making it suitable for hardware implementation.

Structural sparsification focuses on modifying the architecture of neural networks to achieve sparsity, such as by reducing the number of layers, altering connection patterns, or adjusting the size of convolutional kernels. The aim during the model design phase is to introduce sparsity to reduce computational and parameter volume. Techniques of structural sparsification include sparse computational kernels and channels and dynamic sparsification. Sparse computational kernels involve designing computation kernels with sparse patterns to reduce multiply-accumulate operations in convolutional operations. Alternatively, removing entire channels or filters reduces the network's width, significantly decreasing computational volume in convolutional layers.

Dynamic sparsification is a newer technique allowing models to dynamically adjust their sparse structure during the training process. Unlike static sparsification methods, dynamic sparsification techniques learn which connections are essential and which can be removed during training, achieving more efficient and adaptable model sparsification. Sparsity learning introduces additional sparse regularization terms or employs specific learning mechanisms, enabling the network to automatically learn the optimal sparse structure during training. Dynamic sparse networks continually adjust connections to find the optimal sparse topology during training.

Pruning techniques are crucial for achieving structural sparsity in neural network models, significantly reducing model parameters and computational complexity. Fine-grained pruning, which imposes no structural constraints during the pruning process, can traverse all weights and remove any unnecessary ones. Han et al. introduced a deep compression fine-grained pruning method that reduced the number of parameters in AlexNet by 97.12% and in VGG-16 by 97.95%, without any loss in accuracy [4]. Yi's dynamic connection pruning method reduced the parameter count of ResNet-50 by 43%, with only a 0.5% drop in Top-1 accuracy [5]. Liu's frequency domain dynamic pruning scheme achieved a 48% parameter reduction in ResNet-18 with less than 1% accuracy loss [6]. Chang's pruning method, based on modified L1/2 regularization, reduced MobileNet's parameters by 40%,

Table 2
Comparison of pruning methods.

Name	Method category	Model	Compression ratio	Realization
Deep compression [4]	Model	AlexNet VGG-16	39× 45×	Performed in three phases: pruning, training quantization, and Hoffman coding
[5]	Model	AlexNet LeNet-5	17.7× 108×	Use greedy mode pruning and will also restore the subtracted important nodes
Frequency-Domain Dynamic Pruning [6]	Model	AlexNet VGG-16	35× 49×	Only important connections are learned, then weights are shared and finally pruned
Modified L1/2 Penalty [7]	Model	LeNet300-100 LeNet-5 ResNet AlexNet VGG-16	66× 322× 26× 21× 16×	Reducing Error Pruning by Increasing Sparsity in Pre-Trained Models
Accelerator-aware pruning [14]	Model	VGG-16	4×	The weight group is pruned and a fixed number of weights are retained in the group after pruning.
Fast convnets [9]	Convolution kernel	–	–	Generalized convolution is used to reduce matrix multiplication and prune convolution kernels in groups
Pruning Filters [11]	Convolution kernel	VGG-16	2.77×	Remove the entire convolutional kernel and its connected feature maps from the network
[12]	Channel	VGGNet DenseNet-40 ResNet-164	8.71× 2.91× 1.55×	By optimizing the scaling factor γ in the BN layer as a measure of channel importance

maintaining high accuracy while decreasing computational cost [7]. However, fine-grained pruning may result in a large number of sparse matrices, which pose challenges for hardware optimization. To address this issue, Wang et al. proposed sparse convolution units, which store only quantized non-zero parameters and their coordinates, reducing the storage demand of sparse matrices by more than 60% [8].

Coarse-grained pruning involves removing entire layers or convolutional blocks from the neural network architecture. Lebedev et al. proposed a generalized convolution method that reduced the computational load by 30%, with only a 2% accuracy drop [9]. Vector-level and kernel-level pruning are more hardware-friendly; Farone et al.'s kernel pruning framework demonstrated superior resource utilization and power efficiency on FPGAs, improving computational performance by 35% [10]. Li et al.'s study showed that by measuring the importance of convolution kernels through the sum of absolute values and pruning entire kernels and their associated feature maps, parameter counts in VGG-16 and ResNet-110 could be reduced by 50% and 40%, respectively, with only a 1.2% decrease in Top-1 accuracy [11]. Liu et al. optimized channel pruning by evaluating channel importance through the scale factor γ in Batch Normalization layers, pruning unimportant channels, and ultimately maintaining 98 of the original accuracy while halving the computational load [12]. Additionally, the channel grouping and pruning method proposed in [13] reduced computational load by 45%, with only a 1.5% accuracy drop on the ImageNet dataset. These studies indicate that coarse-grained pruning can significantly simplify model complexity and is more suitable for hardware acceleration, making it ideal for resource-constrained embedded systems and edge devices (see Table 2).

3.1.2. Model structure optimization

Model structure optimization occupies a central position in the field of deep learning, aiming to enhance model performance through adjusting and improving the architecture and parameter configuration of neural networks. This process involves meticulous adjustments to several key components, including the number of network layers, the number of neurons per layer, the type of activation functions used, the choice of loss functions, and the variety of optimizers. These adjustments are intended to achieve several key objectives: to increase the model's predictive accuracy, reduce the likelihood of overfitting, decrease the time required for model training, and optimize for specific application scenarios. When undertaking model structure optimization, researchers must consider the balance between model complexity and performance. Overly complex models, while potentially increasing accuracy, may also lead to overfitting and increased computational costs. Conversely, overly simplistic models may fail to capture the complex features of the data. Thus, the process of model structure optimization is also a quest to find the optimal balance point.

1. Knowledge Distillation

Knowledge distillation, as an effective model compression and acceleration technique, aims to transfer knowledge from a complex, larger model (teacher model) to a simplified, smaller model (student model). This process enables the student model to retain prediction accuracy while reducing computational complexity and increasing operational speed. This method mimics human learning by extracting the essence from complex knowledge and teaching it in a simplified manner for efficient learning.

The concept of utilizing knowledge transfer for model compression was first proposed by Buciluǎ et al. who demonstrated the conversion of complex ensemble models into smaller, faster models without significant performance loss [15]. Following this, Ba and Caruana expanded on the idea, showing that complex functions learned by larger Deep Neural Networks (DNNs) could be replicated by smaller, shallower networks, achieving similar accuracy with an equal number of parameters [16]. Hinton et al. further developed the theory of knowledge distillation by introducing a temperature parameter T, drawing from the distillation process in chemistry to purify information, thereby optimizing the learning process of the student model [17].

In subsequent research, Chen et al. within the Net2Net technique, noted that knowledge distillation could significantly reduce the training workload and accelerate training speed through function-preserving

Table 3
Knowledge Distillation Methods Comparison.

Method	Key technology	Main feature
Knowledge transfer	Model compression	Transform complex integration models into smaller models without losing significant performance
Deep network knowledge transfer	Knowledge transfer of deep neural networks (DNNs)	Small shallow networks learn complex functions
Knowledge distillation	Temperature parameter T	Purify the information and optimize the learning process of student model
Net2Net	Function preserving transformation	Reduce training workload and speed up training
Single target detection knowledge distillation	Knowledge transfer	Focus on individual pedestrian detection
DistilBert	Knowledge distillation is applied to BERT model	Simple and effective knowledge distillation application
Channel feature transfer	Feature graph channel exponentiation	Narrow the distance between teacher model and student model
Soft label binding	Soft labels combine with hard labels	Improve model generalization ability
Group learning	Multiple student models learn from each other	Dynamic learning mechanism, improve classification accuracy

transformations, enabling knowledge transfer from large networks to small networks [18]. This technique's application was extended to multi-object detection. In contrast to the work of Chen et al. Shen et al.'s research focused on single pedestrian detection scenarios [19, 20]. Sanh demonstrated how the concept of knowledge distillation could be applied effectively and straightforwardly to the BERT model with DistilBert, further broadening the application scope of knowledge distillation techniques [21].

Zagoruyko proposed a new method of knowledge transfer by performing power operations followed by addition on feature planes across different channels in feature maps. This effectively narrowed the gap between the attention mechanisms of teacher and student models, enhancing the efficiency of knowledge transfer [22]. Addressing the issue of overfitting, Yang showcased the advantages of using a combination of soft and hard labels during the teacher model's training process in improving model generalizability [23]. Zhang further explored a collective learning strategy, where multiple student models learn and guide each other during the training process, demonstrating the potential value of dynamic learning mechanisms among student networks in enhancing classification accuracy [24] (see Table 3).

2. Structural Re-parameterization

The application of Structural Re-parameterization in various Neural Network Model has demonstrated significant improvements in performance and computational efficiency. For instance, the ACNet network, proposed by Dr. Ding Xiaohan's team, replaces traditional KxK convolutions with a summation of three parallel branches (KxK, 1xK, Kx1), resulting in a 1.2% increase in Top-1 accuracy on the ImageNet dataset while reducing computational costs by 30% [25]. Additionally, the Diverse Branch Block (DBB) module, as a versatile basic network component, achieves higher model flexibility and computational efficiency through equivalent transformations between different structures. Experiments on the CIFAR-10 dataset showed that the DBB module increased the Top-1 accuracy of ResNet-56 by 2.1% without increasing the parameter count [26]. These results indicate that Structural Re-parameterization can effectively enhance model accuracy and computational efficiency, making it suitable for training and inference on large-scale datasets.

The RepNAS and ResRep models, based on Structural Re-parameterization, further demonstrate its value in Neural Architecture Search (NAS) and model pruning. RepNAS achieved a 2.5% performance improvement on the NASBench-101 benchmark dataset while reducing computational costs by 35% by dynamically selecting and retaining important branches [27]. The ResRep model maintains network output consistency by inserting 1 × 1 convolution layers after pruning, achieving a Top-1 accuracy drop of only 0.8% on ImageNet with a 50% pruning rate for VGG-16 [28]. Additionally, the RepMLP model enhances the global and local features of MLP by incorporating convolution operations, resulting in a 1.4% improvement in test accuracy on the MNIST dataset [29]. RepVGG simplifies the VGG-style architecture through Structural Re-parameterization, achieving a 40% increase in inference speed while maintaining model accuracy [30]. These experimental results further validate the immense potential of Structural Re-parameterization in optimizing model design and improving computational efficiency, particularly in resource-constrained environments such as FPGA-based deep learning model acceleration (see Table 4).

3. Lightweight Models

Model lightweighting is a key strategy for enhancing inference speed and reducing deployment costs in deep learning. This technique is primarily achieved through pruning and designing compact models. Pruning algorithms create sparse models by eliminating less important parameters in the model, whereas the design of compact models aims to reduce the model's storage space and computational requirements through compact parameter representation.

A common lightweighting method involves substituting large kernels in convolutional layers with smaller kernels, thus selectively creating sparse neural network models. For instance, replacing a single 5 × 5 kernel with two 3 × 3 kernels reduces the number of weight parameters from 25 to 18. Research indicates that constructing novel structures with fewer parameters can achieve equivalent computational results while utilizing fewer parameters.

SqueezeNet is designed to achieve high accuracy with as few parameters as possible. In SqueezeNet, most 3 × 3 convolutional kernels are replaced by 1 × 1 kernels, and the number of input channels to the 3 × 3 convolutions is accordingly reduced [31]. Moreover, MobileNet, designed for mobile devices, introduces the concept of depthwise separable convolutions. Compared to SqueezeNet, MobileNet achieves higher accuracy with a lower number of parameters [32]. ShuffleNet, proposed by the Face++ team, focuses on improving information flow across feature channels through channel shuffle operations, reducing the computational complexity of pointwise convolutions [33].

The Inception model, also known as GoogLeNet, was proposed by Christian Szegedy et al. in 2014 to better utilize the computational resources within the network. Inception V1 reduces computational costs by adding additional 1 × 1 convolutional layers before the 3 × 3 and 5 × 5 convolutional layers to limit the number of input channels [34]. Inception V2 further optimizes the model by introducing Batch Normalization and replacing the 5 × 5 convolutional layers in

Table 4
Comparison of Structural Re-parameterization methods.

Method/Model	Key technology	Experimental result
ACNet	The traditional KxK convolution is replaced by a sum operation of three parallel branches (KxK, 1xK,Kx1)	Top-1 accuracy is improved by 1.2%, and computing costs are reduced by 30%
Deverse Branch Block(DBB)	Equivalent transformations between different structures	2.1% improvement in Top-1 accuracy for ResNet-56
RepNAS	Dynamically select and retain important branches	Improved performance by 2.5% and reduced computing overhead by 35%
ResRep	After pruning, 1 × 1 convolution layer is inserted to maintain network output consistency	When the pruning rate is 50%, the Top-1 accuracy decreases 0.8%
RepMLP	The convolution operation is combined to enhance the global and local MLP	Test accuracy improved by 1.4%
RepVGG	Structure reparameterization simplifies VGG architecture	Inference speed increased by 40%, maintaining model accuracy

V1 with two 3 × 3 convolutional layers [35]. Inception V3 accelerates computation and adds network nonlinearity by transforming some n*n convolutional layers into a combination of 1*n and n*1 layers, reducing the risk of overfitting [36]. Inception V4 introduces dedicated reduction blocks and residual connections, showing an improvement of over 1% in error rate over V3 with similar computational costs [37]. The ESPNet method, proposed by Mehta et al. is a fast, low-power, and low-latency network structure. By introducing an efficient spatial pyramid (ESP) module, which decomposes standard convolution into a spatial pyramid convolution and pointwise convolution [38], ESPNetv2, an extension, demonstrates better generalization capabilities than ShuffleNetV2 [39]. Additionally, SlimNets combine knowledge distillation with pruning among other compression techniques to achieve a model size 85 times smaller than the original model [40].

In the research on deep learning model acceleration technologies for FieldProgrammable Gate Array(FPGA)platforms, the design philosophy of light-weight models is crucial for optimizing model performance. Given the relatively limited computational resources and storage capabilities of FPGAs, adopting lightweighting techniques can effectively reduce resource consumption and enhance model operational efficiency and response speed. Thus, the aforementioned lightweight models and their design methods hold significant reference value and potential for practical application in the acceleration of Neural Network Model on FPGA platforms (see Table 5).

3.1.3. Low-rank approximation

The unprecedented rate at which data is being generated within Neural Network Model has made them a favored choice in AI applications, owing to their capability to extract features from vast datasets. However, the increasing depth and parameter size of neural networks developed for higher accuracy not only consume substantial computational resources but also risk model overfitting, thereby diminishing the models' generalization ability. Given the constraints of modern hardware platforms' memory and processing capacities, dimensionality reduction techniques have emerged as one of the effective means to optimize model performance. The application of online feature selection and low-rank approximations, among other dimensionality reduction techniques, can significantly decrease the number of model parameters and computational complexity. This enables the efficient acceleration of Neural Network Model on FPGA platforms while ensuring model performance.

Tommasel and Godoy's method of online feature selection exemplifies such a technique, utilizing the social and contextual information from short texts generated on social networks to effectively simplify data [44]. This method not only reduces data dimensions but also preserves valuable information for classification tasks, thereby enhancing data processing efficiency and model accuracy under resource constraints.

Low-rank approximation is another widely applied dimensionality reduction technique in Neural Network Model. This method is based on the observation that weight matrices or tensors in models can often be approximated by matrices of a lower rank with minimal impact on model performance. Practically, a large weight matrix with dimensions m×n and rank r can be replaced by matrices of smaller dimensions. Denton proposed an effective method to reduce computational load through low-rank approximations of convolutional kernels and clustering schemes, exploiting redundancies in convolutional filters. This method achieved a twofold acceleration of convolutional layer computations with only 1% decrease in classification accuracy [45].

For research on accelerating Neural Network Model on FPGA platforms, these dimensionality reduction techniques are particularly important. FPGAs offer strong hardware support for accelerating Neural Network Model with their flexible programmability and efficient parallel processing capabilities. However, the limited resources of FPGAs require models to be highly efficient in their computational operations.

In summary, dimensionality reduction techniques play a crucial role in the acceleration of Neural Network Model on FPGA platforms. By effectively reducing the complexity of data and models, these techniques provide feasible solutions for the efficient acceleration of complex Neural Network Model on resource-constrained hardware platforms. Singular Value Decomposition (SVD) is a common and popular factorization scheme for reducing the number of parameters. SVD decomposes the original weight matrix into three smaller matrices, replacing the original weights. The method of Singular Value Decomposition (SVD) introduced by Golub and Reinsch, by decomposing any matrix into the product of three specific matrices, effectively reduces the model's parameter count, aiming to reduce the number of parameters and enhance model efficiency. Principal Component Analysis (PCA) proposed by Jolliffe, through a linear transformation, compresses data and reduces dimensions, enhancing data processing efficiency. Kernel Principal Component Analysis (KPCA) by Schölkopf and Smola utilizes kernel tricks to handle nonlinear data, finding optimal low-dimensional embeddings in high-dimensional feature spaces to optimize data representation. The method of Randomized Singular Value Decomposition (rSVD) by Halko et al. accelerates the computation of SVD through a random algorithm, particularly suitable for large-scale matrices, effectively reducing computation time and resource consumption. Non-negative Matrix Factorization (NMF) by Lee, decomposing matrices into the product of two non-negative matrices, reveals the intrinsic structure of data, widely used in image analysis and text mining. CUR Decomposition by Mahoney, approximating the original matrix by selecting specific columns and rows, maintains data interpretability, optimizing the data analysis process. Tensor decomposition, such as CP Decomposition and Tucker Decomposition proposed by Kolda, breaks down high-dimensional data into products of lower-dimensional or sparse structures, suitable for compression and feature extraction of

Table 5
Lightweight Model Comparison.

Name	Participant numbers	ImageNet Top-1 Correct Rate	Specificities
SqueezeNet [31]	4.8 MB	57.5%	Replacing a 3 × 3 convolutional kernel with a 1 × 1 convolutional kernel
MobileNetV1 [32]	4.2 MB	70.6%	Replacing the standard convolutional layers in VGG with depth-separable convolutions
MobileNetV2 [41]	3.4 MB	72%	Designed structure for inverting residuals
MobileNetV3-small [42]	2.9 MB	67.4%	Optimized network architecture with NAS network search structure
ShuffleNet [33]	1.92 MB	70.9%	Use of channels to disrupt operations to compensate for the exchange of information between subgroups
ShuffleNetV2 [43]	5.96 MB	75.4%	Splitting the input features into two parts achieves the effect of feature reuse
InceptionV1 [34]	5 MB	69.8%	Tapped into the role of 1 × 1 convolution kernels
InceptionV2 [35]	11.2 MB	71.8%	Batch Normalization was proposed to replace the 5*5 convolution in V1 with two 3*3 convolutions
InceptionV3 [36]	23.8 MB	78.8%	Turning an n*n convolution into a two-layer cascade of 1*n and n*1
InceptionV4 [37]	42.6 MB	77.9%	Introduced specialized reduced blocks and residual connections
Inception-ResNet V2 [37]	55.8 MB	80.1%	Combining Inception Architecture and Residual Connectivity
ESPNet [38]	–	–	Point-by-point convolution and null convolution were used
ESPNetV2 [39]	1.12 MB	69.5%	Replace point-by-point convolution with grouped point-by-point convolution and then replace the computationally intensive null convolution with depth-separable null convolution

multidimensional data. Low-Rank Matrix Recovery (LRMR) by Candès, assuming the data matrix is of low rank, recovers the complete matrix from partially observed data, used for data recovery and denoising. The low-rank approximation of Restricted Boltzmann Machines (RBM) proposed by Sutskever, Hinton, and Taylor achieves by reducing the number of units in the hidden layer, maintaining the model's expressive power while reducing model complexity (see Table 6).

3.1.4. Model quantization

In the research and application fields of deep learning, model quantization stands as a key software optimization technique. By reducing the model's storage and computational requirements under a controlled precision loss premise, it enables the efficient acceleration of Neural Network Model on hardware resources-limited platforms, such as FPGA (Field-Programmable Gate Array). Quantization chiefly achieves this by establishing a mapping relationship between high-precision floating-point numbers and low-precision fixed-point numbers, aiming to reduce hardware resource consumption while striving to minimize the error introduced during the quantization process to ensure model performance.

Fixed-point and floating-point quantizations represent two fundamental quantization strategies. Fixed-point quantization employs integers to represent weights and activation values, constrained by bit width; whereas, floating-point quantization maps these values to smaller floating-point representations, commonly 16 or 8 bits. Although theoretically, floating-point quantization could offer higher computational accuracy, fixed-point operations have a comparative advantage in power consumption and resource cost due to their simplified operational logic, thus widely applied in the design of hardware accelerators [46–48].

Linear quantization, being the most prevalent quantization method, has seen mature application in the industry. The 8-bit linear quantization scheme, in particular, has garnered attention for significantly reducing model size and computational volume while maintaining computational accuracy. Research by Chen et al. successfully demonstrated significant resource savings on hardware through compressing 32-bit floating data to 8-bit fixed data [49]. Similarly, Dettmers' development and testing of an 8-bit approximation algorithm validated its effectiveness in compressing gradients and activation values [50, 51]. Furthermore, Gupta et al.'s adoption of 16-bit fixed-point representation effectively reduced memory requirements and floating-point operations in CNN training without significant loss in classification accuracy [52].Dynamic quantization offers a more flexible quantization strategy for different application scenarios, optimizing performance by dynamically adjusting precision. An automatic quantization process proposed by Qiu et al. could adjust precision to 4 or 8 bits according to the dynamic range of data, thereby effectively enhancing quantization performance [53].

Binary quantization and ternary quantization, as special quantization strategies, have shown unique advantages in model compression and acceleration with their extreme approaches. Binary quantization, constraining weights and activation values within [1, −1], although resulting in considerable precision loss, demonstrated significant speed and energy efficiency advantages on platforms like FPGA [54]. In contrast, ternary quantization, by adding an additional digit to represent zero weights, not only prunes network connections to reduce computation but also significantly reduces inference latency while improving model accuracy [55,56].

Beyond traditional linear quantization methods, nonlinear quantization has become a research hotspot due to its better capture of weight

Table 6
Comparison of low-rank approximation methods.

Algorithms	Innovation	Areas of application
SVD	Decomposition into three matrix products reduces the number of parameters	Signal processing statistics
PCA	The data is transformed to a new coordinate system to achieve dimensionality reduction.	Data preprocessing feature extraction
KPCA	Kernel techniques to handle nonlinear data and optimize representations	Nonlinear data dimensionality reduction
rSVD	Stochastic algorithm acceleration for large matrices	Big data processing
NMF	The decomposition into two nonnegative matrices reveals the structure	Image analysis text mining
CUR	Pick specific columns and rows to approximate the original matrix keeping explanatory	Data analysis
Tensor decomposition	Multiple low-dimensional or sparse structures are multiplied to compress the data.	Multi-dimensional data compression feature extraction
LRMR	Assume low-rank recovery of the complete matrix and data recovery	Data recovery Data denoising
Low-rank approximation of Restricted Boltzmann Machine	Reduce complexity by reducing hidden layer units	Optimization of model expressivity
Group Sparse Regularization	Introducing group sparsity to optimize structure	Feature selection model optimization

distribution characteristics. Typical nonlinear quantization methods include logarithmic quantization, which effectively enhances performance density by converting multiplication operations into more efficient shift operations [57,58]. Additionally, Yang et al.'s differentiable quantization method based on a linear combination of multiple Sigmoid functions, and Miyashita's optimization of weight and activation value distribution through logarithmic quantization, both demonstrated the potential of nonlinear quantization in enhancing quantization accuracy and model performance [59,60] (see Table 7).

3.2. Hardware-level optimization

Customized design for deep learning on FPGA represents a significant research area. It focuses on achieving enhanced computational performance and energy efficiency through optimizations in hardware architecture, data management, algorithms, and power management. Data optimization can improve data access and computational efficiency through the use of faster memory or caching technologies, optimizing data layouts, and reducing data transmission. Optimization of computation cores involves refining operations such as convolutional computations, matrix multiplications, and other common computational tasks to decrease the amount of computation and energy consumption. Hardware architecture optimization can enhance computational performance and energy efficiency by increasing the number and parallelism of computational units, enhancing the efficiency of storage units and data pathways, and improving clock management (see Table 8).

3.2.1. Data flow optimization

Customized design for deep learning on FPGA represents a significant research area. It focuses on optimizing aspects such as hardware architecture, data management, algorithms, and power management to achieve more efficient computational performance and energy efficiency. Optimization of data can enhance data access and computational efficiency through the use of faster memory or cache technologies, optimized data layout, and reduced data transmission. Optimization of computational cores can address the crucial issue of data transmission in high-performance computing, a time-consuming component of computational tasks. In practical applications, data transmission involves storage, reading, and transferring of data, consuming computational resources and affecting performance and energy efficiency. The latency of data transmission also poses a challenge, especially in distributed computing environments where data must be transmitted over networks. In typical edge devices with limited on-chip resources due to embedded implementations, frequent data exchanges with external memory sometimes consume more energy than computation [65].

Data reuse techniques play a pivotal role in accelerating Neural Network Model, particularly in computationally intensive operations of Convolutional Neural Networks (CNNs), such as convolution. By effectively reusing data already loaded onto on-chip storage, this technique can significantly reduce the number of data transmissions, thereby enhancing data processing efficiency and reducing energy consumption. On FPGA platforms, the application of data reuse techniques is especially critical due to the relatively limited computing resources and storage capabilities of FPGAs, necessitating optimized algorithms and hardware designs for efficient data processing.

Convolution operations inherently involve four nested loops: traversing channels of the output feature map, traversing each element of the output feature map, traversing elements of the convolution kernel, and traversing elements of the input feature map. These loops form the basis of convolution operations, and the computational complexity and volume of data access require effective management and optimization of computational resources.

To maximize the parallelism of Multiply-Accumulate (MAC) operations in convolution, loop unrolling techniques are often employed. Loop unrolling increases the parallelism of data processing but also demands more storage resources. Therefore, balancing the increased

Table 7
Comparison of quantitative methods.

Name	Quantitative method	Model-specific	Realization details
DaDianNao [49]	8-bit	Convolutional neural network	Compression of 32-bit floating-point data to 8-bit fixed-point data
8-BIT APPROXIMATIONS [50]	8-bit	Deep neural network	8-bit approximation algorithm
[52]	16-bit	Deep neural network	Applying random rounding schemes when operating on indeterminate points
[53]	4-bit and 8-bit	Convolutional neural network	Quantification of dynamic accuracy
22FP-BNN [54]	Two-valued	Deep neural network	Over-bit level XNOR and shift operations eliminate bottlenecks in multiplier operations
XNOR-Net [61]	Two-valued	Convolutional neural network	Finding the best approximation of a convolution using binary operations
Ternary Neural Network [55]	Three-valued	Deep neural network	Thresholds for weight trinomialization are merged into closed form using truncated Gaussian approximation
[56]	Three-valued	Convolutional neural network	Training Ternary Quantization
Lognet [57]	Log2	Deep neural network	Replacing Digital Multipliers with Log2 Quantization
[59]	Sigmoid	Convolutional neural network	Approximate quantization function generated by replacing the step function with a Sigmoid function with temperature parameter T

Table 8
Custom Design Hardware Optimization Comparison.

Dimension	Design Purpose	Design Methodology	Specificities	Field Of Research	Representative Studies
Data Optimization	Efficient data storage and access to meet computing requirements	Optimization of storage media and storage methods, optimization of data transmission methods	1. Optimized for data 2. Strong generalizability	1. Data storage 2. Data transmission 3. Data flow	[62–64]
Computational Kernel Optimization	Achieve more compute units and higher peak performance	Load balancing by disaggregating or fusing computational cores according to the actual situation	1. Reconfigurability 2. Parallelism-oriented design of computational cores	1. Fusion calculation kernel 2. Decomposition calculation kernel	[65–67]
Hardware Architecture Optimization	Adaptation to specific application scenarios and performance requirements	Hardware gas pedals, instruction set optimization, architectural innovation, etc	1. Consider memory bandwidth 2. Consider hardware resource utilization	1. Single-engine architecture 2. Streaming architecture	[62,68,69]

parallelism from loop unrolling with efficient FPGA resource utilization becomes a critical consideration in design optimization. Zhang et al.'s research indicates that excessive loop unrolling can lead to wastage of FPGA resources, reducing system performance, thus careful selection of unrolling parameters is required to optimize resource utilization [66].

Addressing resource limitations, an effective strategy is to introduce data reuse mechanisms in hardware design. By storing read data in FPGA on-chip caches and reusing it in subsequent MAC operations, the number of data reloads can be reduced, enhancing data processing efficiency. Rahman A et al. demonstrated efficient data reuse by unrolling loops of the convolution kernel and input elements and utilizing a "local cache" mechanism in FPGA to cache input data blocks [70]. This approach not only reduces the number of data transmissions but also increases computational parallelism.

Moreover, loop pipelining is another key technique to enhance data reuse rates and accelerate deep learning model performance. By unfolding the multidimensional loops of convolution operations into a one-dimensional loop and employing loop pipelining to decompose the convolution operation into multiple stages, with pipeline registers introduced between each stage, parallel execution of computational units can be achieved. Dong et al.'s work showcased data reuse through loop caching and optimized computational efficiency with pipeline technology [67].

Data reuse techniques play a crucial role in accelerating Neural Network Model on FPGA platforms. Through carefully designed loop unrolling strategies, data reuse mechanisms, and efficient loop pipeline designs, data processing efficiency can be significantly improved, energy consumption reduced, and FPGA resource utilization optimized. The combination of these techniques provides effective methodological support for achieving efficient acceleration of Neural Network Model on resource-constrained FPGA platforms. Optimization methods that include compressing data and optimizing data layouts reduce storage space, lower data access latency, and enhance parallelism. Besides reducing model scale and parameter count, employing effective data storage and access methods can meet computational demands. Data storage and access efficiency are among the critical factors affecting machine learning model performance. Optimizing operations such as convolution, matrix multiplication, and other common computational tasks can reduce computational workload and energy consumption. Hardware architecture optimization, through increasing the number of computational units and parallelism, enhancing the efficiency of storage units and data pathways, and improving clock management, can enhance computational performance and energy efficiency.

2. In the implementation and optimization processes of Neural Network Model, data compression technology serves as a crucial method,

effectively reducing the storage and transmission requirements of models. Common data compression techniques include Huffman coding, Lempel–Ziv-Welch (LZW) encoding, Compressed Sparse Row (CSR) encoding, and Compressed Sparse Column (CSC) encoding. Each of these technologies offers distinct advantages and limitations across various application scenarios, where choosing an appropriate compression strategy can significantly enhance data processing efficiency and reduce the consumption of hardware resources.

In optimizing data layout, particularly for handling sparse data structures in deep learning, such as the attention mechanism layer in Transformer models, Li et al. proposed a novel data storage format—Packed Sparse Column (PSC). The PSC format, leveraging sparse matrix compression technology, is specifically optimized for input vectors containing a large number of zero elements. It begins by removing all zero elements from the input vector, retaining only the non-zero elements, which are then compressed into a dense vector. Subsequently, this dense vector is divided into multiple column segments, each containing a fixed number of non-zero elements, to accommodate the access characteristics of block RAM in FPGAs, thereby achieving more efficient data access [63].

The introduction of the PSC format holds significant implications for accelerating deep learning models on FPGA platforms, especially models based on the Transformer architecture. By reducing the storage of zero elements and optimizing the access pattern of non-zero elements, the PSC format can lower storage demands and enhance the efficiency of data access while ensuring computational accuracy. Moreover, this optimization also aids in improving the utilization of FPGA resources, including reducing memory bandwidth requirements and lowering power consumption, thus providing an effective acceleration solution for deep learning models on FPGA platforms.

3. In the realm of efficient acceleration of Neural Network Model, the "memory wall" issue, stemming from the mismatch in speed between processors and memory units, poses a significant challenge. This issue arises when the access speed of memory units fails to keep pace with the processing speed of computing units, making data exchange a bottleneck for system performance. Although optimizing data storage formats can somewhat reduce storage space requirements, it does not fundamentally address the memory wall problem. Consequently, researchers have proposed two main solutions: one is to enhance memory access speed through the adoption of advanced storage technologies; the other is to reduce the number of memory accesses by optimizing algorithms and data structures, as well as adopting in-memory computing architectures.

Firstly, employing faster storage technologies, such as high-bandwidth memory and 3D-stacked DRAM, can significantly improve memory access speed, thereby alleviating the latency and energy consumption issues associated with memory access [62]. To overcome the limitations of fixed data storage patterns in existing memory technologies, Akin et al. introduced the concept of DataMover, a novel hardware accelerator. DataMover can dynamically reorganize data within 3D-stacked DRAM, including data rearrangement, recombination, and pruning, to enhance the efficiency of memory access. Moreover, this accelerator can also reduce the power consumption during memory access. The memory hierarchy design based on DataMover enables dynamic reorganization and rearrangement of data in memory and stores these data in a new high-speed cache layer, thus optimizing data access patterns [71].

Secondly, strategies that reduce the number of memory accesses through the optimization of algorithms and data structures effectively decrease the program's reliance on memory. Compute-In-Memory (CIM) architecture, as an emerging technology, integrates computing logic within memory to achieve a seamless integration of storage and computation. This architecture can significantly mitigate or even eliminate the discrepancy between storage and computing units, addressing the memory wall issue at its root. Yin et al. explored the application of volatile memory in supporting in-memory computation and how these technologies can achieve more efficient in-memory computing [64]. On FPGA platforms, with their abundant distributed on-chip resources, efficient in-memory computing architectures can be constructed. For instance, Irfan proposed a novel TCAM architecture based on distributed RAM, which, by distributing memory blocks across different storage nodes, avoids the global memory bottleneck inherent in traditional TCAMs, achieving significant performance improvements [72].

Addressing the memory wall problem in the acceleration of Neural Network Model on FPGA platforms requires a comprehensive consideration of both strategies: employing advanced storage technologies and reducing the number of memory accesses. By optimizing memory data organization with hardware accelerators such as DataMover and implementing in-memory computing architectures, memory access speed and computational efficiency can be effectively enhanced.

3.2.2. Parallel computing architecture

FPGA-based parallel computing architectures facilitate the efficient acceleration of complex algorithms by distributing computational tasks across multiple logic units on the FPGA. The flexibility and reconfigurability of FPGAs offer unique advantages for parallel computing, allowing developers to customize hardware logic according to specific task requirements, thereby optimizing computational performance and resource utilization. FPGA-based parallel computing architectures can be classified based on their parallel strategies and implementation methods. One category is data-level parallelism, which involves performing the same operation on different pieces of data at the same time. Another category is task-level parallelism, which involves the simultaneous execution of different computational tasks. The third category is pipeline parallelism, which accelerates computation by decomposing the process into multiple sequential stages, with each stage processing different data or tasks.

1.Data-level parallelism is a key concept in parallel computing, particularly when handling large datasets. It accelerates the computation process by simultaneously performing the same operations on multiple data items. In FPGA-based parallel computing architectures, data-level parallelism leverages the parallel processing capabilities of FPGAs to significantly enhance the speed of data processing tasks.

For large-scale matrix multiplication operations, traditional serial processing methods are inefficient and unable to meet real-time computation requirements. Implementing matrix multiplication on FPGAs typically involves dividing large matrices into smaller sub-matrix blocks, which can be processed in parallel. Multiple processing units within the FPGA, such as DSP blocks, are utilized to compute the products of these sub-matrices in parallel. By designing effective data flow and storage strategies, data can be efficiently transferred between processing units, reducing access latency. Furthermore, employing pipelining techniques can further increase the computation throughput, allowing each processing unit to simultaneously process different data blocks at various stages. Kestur S. proposed a matrix algorithm that achieved high-speed computation of matrix multiplication operations, demonstrating the performance advantages of FPGAs in parallel processing compared to CPUs and GPUs.

Given the wide application of Fast Fourier Transform (FFT) in signal processing and communication fields, there is a need for efficient algorithms to support large data volumes and high-speed processing. The FFT process can be decomposed into multiple levels of butterfly operations, each of which can be executed in parallel. On FPGA, these butterfly operations can be mapped to different logic units, enabling parallel processing. By pipelining these butterfly operations, more computations can be completed within each clock cycle, significantly enhancing the processing speed of FFT. Garrido M proposed an FFT method that, by incorporating pipeline technology, enhances computational efficiency, providing an effective real-time FFT computation solution for signal processing and communication systems.

In the realm of deep learning, especially in applications of Convolutional Neural Networks (CNNs) in fields such as image and speech

processing, the demand for extensive convolutional computations poses a challenge for traditional methods to meet efficiency requirements. Zhang et al. proposed a high-speed acceleration algorithm for CNNs, which accelerates convolutional operations on FPGAs primarily through the parallel execution of multiple convolution operations. This approach involves mapping input data (e.g., images) and convolution kernels onto different logical units of the FPGA, followed by parallel computation of their convolutions. Additionally, a data reuse strategy is employed to reduce repetitive access to input data, enhancing overall computational efficiency.

Sorting is a common task in computing, where traditional serial algorithms prove inefficient for large-scale data sorting. Efficient parallel sorting networks on FPGA are essential, utilizing parallel comparisons and swaps to expedite the sorting process. Parallel sorting algorithms on FPGAs, such as bitonic sorting networks, achieve sorting through parallel comparison and element swapping. They also leverage logic units to execute multiple comparisons and swaps in parallel, quickly organizing data. To optimize performance, the sorting process can be designed as a multi-stage pipeline structure, with each stage completing a portion of the sorting task. Furthermore, a well-planned data input and output strategy can minimize data transmission delays, enhancing sorting efficiency. The approach by D. Ziouzios et al. significantly accelerates the sorting process by parallelizing sorting operations on FPGAs.

2. The emergence and development of Task-Level Parallelism in FPGA-based systems primarily aim to fully leverage the parallel processing capabilities of FPGAs, enhancing the performance of multi-task computing applications. This parallel strategy effectively addresses the issue where single-task processing fails to utilize FPGA resources comprehensively.

Dynamic task scheduling focuses on adjusting the priority and resource allocation of task execution dynamically based on the runtime state of the system, such as the task queue and resource utilization rates. Dynamic task scheduling algorithms typically require monitoring the system's current state, including the resources occupied and the list of tasks pending execution. Then, based on certain strategies, such as prioritizing shortest completion time or lowest resource consumption, these algorithms dynamically adjust the order of task execution and allocate resources. Innovations in this field are primarily concentrated on developing more intelligent scheduling strategies and algorithms to achieve better resource utilization rates and shorter task completion times. For example, some studies have introduced machine learning methods to predict the execution time and resource requirements of tasks, thereby enabling more accurate task scheduling and resource allocation.

The Resource Sharing Strategy is pivotal in FPGA environments where resources, such as DSP blocks and LUTs, are limited. This strategy aims to design a mechanism that allows for the sharing of FPGA computational resources among different tasks without causing resource conflicts. This typically involves scheduling resource access to ensure each task can obtain sufficient resources when needed. Developing efficient resource scheduling algorithms and sharing mechanisms is crucial to maximize the utilization of FPGA resources. The Resource Fair Sharing (RFS) method reduces idle time caused by resource waiting through optimizing resource allocation algorithms.

Multi-Task Pipelining, within the context of FPGA data processing applications, leverages pipeline technology to significantly enhance data processing throughput and reduce processing latency. Organizing different tasks into a pipeline format enables efficient parallel data processing in complex applications. The design of multi-task pipelines typically involves decomposing the entire computation process into multiple stages (or tasks), with each stage completing a specific computational task. Peng and colleagues introduced a dynamic adjustment mechanism that adaptively adjusts the pipeline configuration based on changes in the data flow, thereby improving overall processing efficiency.

Heterogeneous Task Parallel Processing addresses the complexity of modern computing tasks, which often comprise sub-tasks of diverse natures, such as data computation and logic control, each with unique computational resource requirements. Efficiently executing these heterogeneous tasks on FPGAs is key to enhancing computational performance. The strategy for heterogeneous task parallel processing aims to map tasks of different natures to the most suitable regions of the FPGA, ensuring they can execute in parallel without mutual interference. Robert's research focuses on designing FPGA architectures that support rapid task switching and resource reconfiguration to accommodate the varying needs of different tasks.

3.Pipeline Parallelism enhances data processing throughput and minimizes latency by decomposing complex computational tasks into a sequence of smaller, more manageable sub-tasks, which are then executed in parallel within an FPGA in a pipelined manner. This approach allows data and tasks to be naturally segmented into multiple consecutive processing stages, significantly improving efficiency in handling computational tasks.

Data Pipelining primarily focuses on the continuous flow of data between processing units. In neural network computation, this means that input data, such as image pixels or sensor readings, can be continuously passed through different processing stages, each performing operations like convolution, pooling, or activation functions. Data pipelining allows for the simultaneous processing of multiple data items, with each processing stage handling a different data item, thus achieving efficient parallel processing. In FPGA implementations of video processing applications, data pipelining is commonly used to process video frames continuously. For example, the processing workflow for a video stream might include steps like frame decoding, color space conversion, filtering, and encoding. By setting up dedicated processing units for each step in the FPGA and sequentially passing video frames through these units, efficient video stream processing can be achieved (see Table 9).

Model Pipelining decomposes the neural network model itself into multiple parts or layers, which are then executed in parallel on different logic units within the FPGA. For instance, a deep convolutional neural network can be decomposed into multiple convolutional layers and fully connected layers, which are then mapped onto different regions of the FPGA for parallel execution. Model pipelining is particularly suitable for large neural network models where the data dependency between different layers is low, allowing for a higher degree of parallelism.

Instruction Pipelining involves decomposing the sequence of computational instructions of a neural network into different execution stages. In FPGAs, this might involve creating dedicated hardware execution units for performing specific arithmetic or logic operations and organizing these operations into a pipeline. In this way, FPGAs can process multiple instructions at different execution stages simultaneously, enhancing instruction-level parallelism. When implementing fully connected layers of a neural network, the core computation is matrix multiplication. Using instruction pipelining in FPGAs, matrix multiplication can be broken down into a series of multiplication and addition operations, which are executed in parallel at different stages of the FPGA. For example, all multiplication operations are first computed, and then the results are accumulated to obtain the final matrix elements. By pipelining multiplication and addition operations, the processing speed of matrix multiplication can be significantly accelerated.

Hybrid Pipelining combines the aforementioned pipelining parallel strategies. In practice, FPGA designers may adopt a combination of data pipelining, model pipelining, and instruction pipelining based on the specific neural network structure and performance goals to achieve optimal acceleration effects. In a complex image recognition system that includes image preprocessing, feature extraction, and classification as the three main steps, Li and others employed a hybrid pipelining strategy. The image preprocessing step (e.g., filtering, edge detection)

Table 9
Comparison of pipeline parallel methods.

Pipeline type	Descriptiond	Typical application	advantage
Data pipeline	The continuous flow of data between processing units	Video frame processing large neural networks	Efficient parallel processing
Model pipeline	Model Layering for Parallel Execution	large-scale neural network	Increased parallelism
Instruction pipeline	The instructions are implemented in stages	Matrix multiplication	Accelerated processing speed
Hybrid pipeline	Combining multiple pipeline strategies	Image recognitionk	Maximizing the acceleration effect

can use data pipelining techniques to process image frames continuously; the feature extraction step (e.g., through convolutional neural networks) can use model pipelining techniques to process different layers; and the final classification step can utilize instruction pipelining techniques to accelerate computation. This hybrid pipelining strategy fully leverages the FPGA's parallel processing capabilities, achieving efficient acceleration of the entire image recognition process. The peak performance of neural network accelerators is largely influenced by the design at the computational unit level. In FPGA chips, due to limited resources, designing smaller computational units can enable more computational units and higher peak performance. Miniaturizing multiply-accumulate units can better utilize FPGA resources. Placing computational units in a tight array can shorten the communication distance between units and reduce communication overhead, thus better leveraging FPGA's bandwidth and latency characteristics.

Alwani found in his research that processing CNN layers generates a large amount of off-chip data transfer, and computational results are stored in off-chip memory, leading to extensive off-chip memory access. To address this issue, he proposed a fused layer architecture that can efficiently utilize local memory to process convolutional data streams. This architecture combines the computation of convolutional and activation layers into one pipeline and adopts a local reuse strategy to reduce memory access frequency. This allows for simultaneous multilayer convolution, with intermediate results directly used for the next layer's computation [73]. Similarly, Zhao and others adopted a fused design, merging convolutional layers, batch normalization layers, and activation function layers into one computational unit, thereby avoiding data movement and memory access between these layers. To support the limited on-chip resources of small FPGAs, the authors used tiling techniques to break down convolutional blocks into smaller blocks and proposed a computational core design method based on the Winograd convolution algorithm. The Winograd algorithm transforms convolution kernels and input feature maps, converting convolution operations into a series of small-scale matrix multiplications. By designing various sizes of computational cores, the authors achieved support for different scales of convolutional operations and optimized the data flow and control logic of the computational cores [74].

Unlike the aforementioned works, Steo and others chose to eliminate the output buffer, sending each layer's output directly to the input buffer. Additionally, they used FIFO channels to reduce on-chip memory usage [75]. Li proposed an end-to-end FPGA-based CNN accelerator that maps all layers onto one chip, allowing different layers to work concurrently in a pipeline structure. They merged multiple convolutional layers and pooling layers into one large convolutional layer and combined multiple fully connected layers into one large fully connected layer. In terms of computational core optimization, the authors used a sliding window method for convolution computation, which decomposes convolution computation into multiple small computational cores and maps them onto FPGA's computational resources [76].

3.2.3. Hardware accelerator and advanced design framework

In the process of accelerating neural networks with FPGAs, hardware optimization design plays a crucial role, primarily aimed at enhancing computational performance, reducing power consumption, and optimizing the use efficiency of FPGA resources. Dedicated hardware acceleration units and dynamic reconfiguration technology, designed for specific neural network operations like convolution and pooling, provide targeted optimization pathways for FPGAs. Custom acceleration modules can specifically enhance the efficiency of operations, while the dynamic reconfiguration ability of FPGAs allows for flexible switching between different network layers, realizing the maximization of resource utilization. These hardware optimization design strategies require a comprehensive consideration of the neural network's architecture, FPGA's resource constraints, and the performance demands of the target application. Through meticulous design and adjustment, the operational efficiency and effectiveness of neural networks on FPGAs can be significantly improved.

1.Hardware accelerator

FPGA accelerators exhibit diversity in hardware architecture design to accommodate various application scenarios and performance requirements. These architectures are typically divided into two main types: single-engine architectures and streaming architectures. Single-engine architectures, suitable for conventional digital logic design, are characterized by their implementation in systolic arrays, an architecture with high parallelism, regular layout, and efficient local communication characteristics. The advantage lies in reducing data transmission needs and supporting high clock frequency operation. For instance, the Caffeine scheme proposed by Zhang et al. is an FPGA accelerator design based on a systolic array structure, which, by integrating into the Caffe deep learning framework, achieves efficient parallel inference of CNNs on FPGAs [68]. Furthermore, Wei proposed a design space exploration strategy for two-dimensional systolic array architectures, transforming CNN convolution operations into matrix multiplications and further decomposing matrix multiplications into sub-matrices, mapped onto the systolic array to achieve high throughput computation [77].

Streaming architectures, mainly aimed at high-performance applications for data-intensive tasks, adopt the design philosophy of implementing each neural network layer as an independent Processing Element (PE) and processing data in a pipelined manner to accelerate overall computation. This architecture requires precise design of PEs to ensure consistent computation time across layers, reducing computational delay. Compared to single-engine architectures, streaming architectures have lower global data transmission requirements but lack flexibility in scaling after being optimized for specific neural network architectures. The DRAMA architecture proposed by Farmahini-Farahani attempts to enhance memory access bandwidth and reduce data movement power consumption by closely connecting more on-chip storage units with computing units and using through-silicon via technology to stack reconfigurable array chips and DRAM [78]. Additionally, Sina Ghaffari and others designed two hardware architectures for deep neural networks: one tailored for small networks, designing specific hardware for each layer; the other reuses hardware designed for each layer, controlled by loops to determine its timing, accommodating varying levels of demand [79] (see Table 10).

2. Hardware-Aware Neural Architecture Search

In the context of real-world applications such as autonomous driving and mobile devices, performance metrics like power consumption

Table 10
Comparison of hardware optimization methods.

Architecture/methodology	Strategy	Flexibilities	Efficiency
Contraction Array	Highly parallel architecture regular layout reduces data transfer supports high frequency	Medium	High
Caffeine	Shrinkage-like architectural design with integrated Caffe framework for CNN parallel inference	Medium	High
Design Space Exploration for 2D Shrinking Display Architecture	The CNN convolution is converted to matrix multiplication and divided into submatrices to be processed in parallel in the shrinking display.	Low	High
Pipeline architecture	Each layer is realized as an independent PE, pipelined to process the data, and needs to be consistent with the computation time of the same layer	Low	High
DRAMA architecture	Stacking on-chip memory cells reduces transfers, and stacking display chips and DRAM using silicon perforation improves broadband.	Medium	High
Dedicated hardware architecture for deep neural networks	Design of two architectures: hardware specific for small networks and hardware reuse through control loops	First low second medium	First high second medium

and computational latency have become key parameters in assessing the practicality of models. Against this backdrop, hardware-aware Neural Architecture Search (NAS) technology has emerged, aiming to find neural network architectures that optimize performance under specific hardware constraints, considering the physical limitations of hardware platforms. Unlike traditional optimization methods that rely solely on software or hardware, hardware-aware NAS technology strives to achieve an optimal balance between model accuracy, latency requirements, flexibility, and speed.

The core of hardware-aware NAS technology lies in optimizing adaptation to various hardware platforms to generate neural network models that meet time efficiency while maintaining the highest accuracy requirements. By analyzing hardware characteristics such as computational capacity, storage capacity, and energy consumption in depth, hardware-aware NAS provides precise and practical guidance for model design, ensuring optimal performance under hardware resource constraints. Successful implementation of this approach relies on a profound understanding of hardware platform operating mechanisms and performance metrics, as well as a comprehensive grasp of neural network model structures and computational demands.

In the field of NAS, researchers have proposed various methods aimed at catering to specific application needs and performance optimization goals. For instance, FBNet [80]employs a tiered search space and implements a differentiable model structure search based on latency metrics, avoiding exhaustive search and independent model training processes while maintaining lower latency. ProxylessNAS [81] significantly shortens training time and improves model accuracy through a single-path super-network and model parameter pruning strategy, without relying on subsets or proxy datasets. Single-Path NAS[96] aims to reduce search costs and model parameter count by searching a hyperparameterized super-network with shared weight kernels, though its search space is somewhat limited. SPOS [82] combines a single-path super-network, uniform sampling, and evolutionary algorithms, decoupling network weights from structures and simplifying super-network training to support complex search spaces. MnasNet[106] utilizes a hierarchical decomposition and blockIn the context of real-world applications such as autonomous driving and mobile devices, performance metrics like power consumption and computational latency have become key parameters in assessing the practicality of models. Against this backdrop, hardware-aware Neural Architecture Search (NAS) technology has emerged, aiming to find neural network architectures that optimize performance under specific hardware constraints, considering the physical limitations of hardware platforms. Unlike traditional optimization methods that rely solely on software or hardware, hardware-aware NAS technology strives to achieve an optimal balance between model accuracy, latency requirements, flexibility, and speed.

The core of hardware-aware NAS technology lies in optimizing adaptation to various hardware platforms to generate neural network models that meet time efficiency while maintaining the highest accuracy requirements. By analyzing hardware characteristics such as computational capacity, storage capacity, and energy consumption in depth, hardware-aware NAS provides precise and practical guidance for model design, ensuring optimal performance under hardware resource constraints. Successful implementation of this approach relies on a profound understanding of hardware platform operating mechanisms and performance metrics, as well as a comprehensive grasp of neural network model structures and computational demands.

In the field of NAS, researchers have proposed various methods aimed at catering to specific application needs and performance optimization goals. For instance, FBNet [80] employs a tiered search space and implements a differentiable model structure search based on latency metrics, avoiding exhaustive search and independent model training processes while maintaining lower latency. ProxylessNAS [81] significantly shortens training time and improves model accuracy through a single-path super-network and model parameter pruning strategy, without relying on subsets or proxy datasets. Single-Path NAS [83] aims to reduce search costs and model parameter count by searching a hyperparameterized super-network with shared weight kernels, though its search space is somewhat limited. SPOS [82] combines a single-path super-network, uniform sampling, and evolutionary algorithms, decoupling network weights from structures and simplifying super-network training to support complex search spaces. MnasNet [84] utilizes a hierarchical decomposition and block partitioning search space, and through reinforcement learning strategies, considers inference latency constraints while balancing inference latency and accuracy. FNAS [85] achieves a rapid, training-free search process through a performance abstraction model and pruning strategy based on reward functions.

Especially for hardware platforms like FPGAs with rich distributed on-chip resources, hardware-aware NAS technology offers a new avenue for optimizing deep learning model acceleration. EDD [86] effectively merges hardware and model parameters by integrating a joint

search space of deep neural network search variables and hardware implementation variables, enhancing model performance. NASCaps [87] employs the genetic algorithm NSGA-II, considering capsule layer emulation support for the first time, providing insights for the deployment of high-accuracy models in edge computing scenarios. APNAS [88] achieves efficient model optimization through weight sharing and reinforcement learning search with a recurrent network controller. HotNAS [89] achieves rapid search through model compression based on a pre-trained model collection.

Partitioning search space, and through reinforcement learning strategies, considers inference latency constraints while balancing inference latency and accuracy. FNAS [85] achieves a rapid, training-free search process through a performance abstraction model and pruning strategy based on reward functions.

Especially for hardware platforms like FPGAs with rich distributed on-chip resources, hardware-aware NAS technology offers a new avenue for optimizing deep learning model acceleration. EDD [86] effectively merges hardware and model parameters by integrating a joint search space of deep neural network search variables and hardware implementation variables, enhancing model performance. NASCaps [87] employs the genetic algorithm NSGA-II, considering capsule layer emulation support for the first time, providing insights for the deployment of high-accuracy models in edge computing scenarios. APNAS [88] achieves efficient model optimization through weight sharing and reinforcement learning search with a recurrent network controller. HotNAS [89] achieves rapid search through model compression based on a pre-trained model collection (see Table 11).

To enhance the adaptability and performance of Neural Architecture Search (NAS) technology on hardware platforms, researchers have explored two key directions to optimize the NAS process for more effective utilization of hardware resources, especially for programmable hardware platforms like FPGAs. These directions include: (1) refining the search space to reduce search complexity, and (2) integrating hardware constraints directly into the search process to generate models that better match specific hardware.

Firstly, strategies for controlling the search space aim to simplify the complexity of NAS without altering the fundamental structure of the search space. For instance, hierarchical search strategies used by FBNet [80] and MnasNet [84] leverage hardware parameters to refine the search range layer by layer, effectively reducing search complexity and enhancing search efficiency. However, this method might fail to discover the optimal neural network model due to inherent limitations of the search space. Similarly, ProxylessNAS [81], Single-Path NAS [83], and SPOS [82] adopt network pruning methods, adjusting the network structure itself to fit hardware parameters. This not only lightens the search burden but also ensures the discovery of efficient network structures.

The second direction involves considering hardware constraints or parameters directly in the search process, incorporating hardware characteristics into the definition of the search space from the beginning. This approach can directly generate a large number of model structures suited to specific hardware, thereby effectively improving search efficiency and model performance. For example, EDD [86], APNAS [88], and NASCaps [87] use reinforcement learning strategies to reduce errors in hardware parameter estimation, while studies like HotNAS [89] base model searches on predefined templates. These strategies, by specifically optimizing hardware parameters, significantly enhance model performance. Additionally, research [91], HURRICANE [90], and [92] propose diverse solutions specifically targeting different hardware parameter constraints, ensuring optimal model performance under varied hardware conditions

3.Advanced framework

In research aiming at accelerating Neural Network Model on FPGA platforms, the application of high-level frameworks and high-level synthesis (HLS) tools is key to enhancing development efficiency and model performance. These tools and frameworks enable researchers to design and implement hardware acceleration schemes using high-level programming languages, thereby optimizing the operational efficiency of Neural Network Model on FPGAs.

Caffe [93], an open-source deep learning framework written in C++ and providing interfaces for Python and MATLAB, has become a popular choice for training and deploying generic neural network models. Xilinx's ML Suite officially recommends and supports the use of Caffe, significantly boosting the efficiency of implementing deep learning solutions on FPGAs. Optimizations for the Caffe framework mainly fall into two directions: first, binding Caffe with other frameworks, such as Caffe-OpenCL, which transforms the Caffe framework to support an OpenCL backend, enhancing cross-platform portability; second, replacing certain functions in Caffe to boost performance, for example, FixCaffe [94] optimizes performance by substituting high-precision floating-point matrix multiplication functions with low-precision fixed-point counterparts.

In the hardware design process, the Vitis suite, as one of Xilinx's most commonly used HLS tools, supports C/C++/OpenCL languages, providing a unified software platform for FPGA design. Vitis greatly simplifies the hardware design verification process through optimized AI hardware libraries and support for MATLAB/Simulink environments. Similarly, Intel's Quartus Prime, a complete development suite for Intel FPGAs, integrates the entire development flow from design input, synthesis, optimization, and verification to simulation. With the widespread adoption of OpenCL, tools specific to OpenCL, like Intel SDK and SDAccel, are extensively applied in FPGA design. For researchers wishing to directly use high-level languages for hardware design, tools based on high-level languages offer the capability to compile model code into hardware description languages. Python, highly compatible with deep learning frameworks, has seen its Python-based hardware description language compilers become increasingly popular. Python Hardware Description Language (PyHDL) [95]and MyHDL [96] showcase how to leverage Python for object-oriented hardware design and directly use it as a hardware description and verification language, simplifying the hardware design process. PyMTL [97] addresses the issue of long simulation times through mixed JIT compilation techniques, while PyRTL [98] provides an effective method to describe digital hardware structures utilizing Python language features.

Other high-level programming languages, such as Java and Ruby, have also been applied to hardware description for FPGAs. JHDL [99], based on Java, employs an object-oriented approach to manage FPGA resources, offering flexible switching between simulation and execution environments. Similarly, rHDL [100], a hardware description language based on Ruby, supports object-oriented design methods (see Table 12).

3.3. Compiler optimization

In the system design of FPGA-based deep learning model acceleration technologies, the choice of hardware design methodology significantly impacts efficiency, performance, and development cycles. Currently, hardware design approaches can be primarily categorized into three types: Register Transfer Level (RTL)-based, High-Level Synthesis (HLS)-based, and hybrid methods.

1. RTL-based Design: Traditionally, hardware design using Verilog and VHDL at the RTL is the cornerstone for describing Very Large Scale Integration (VLSI) systems. This method allows developers to control hardware behavior with high precision, thus achieving high-performance designs. However, the complexity and lengthy development cycles of RTL design have limited its application, especially in projects requiring rapid iterations of network models.

2. HLS-based Design: To alleviate the complexity of traditional RTL design and shorten development cycles, High-Level Synthesis (HLS) technology emerged. HLS tools enable the generation of RTL-level abstractions for target FPGA devices using high-level programming languages such as C/C++ or MATLAB Simulink and graphical modeling. By utilizing libraries based on high-level languages and OpenCL, the

Table 11
Comparison of search methods for hardware-aware neural architectures.

Method Name	Search Space	Search Strategy	Dominance	Inferior	Performances
FBNet [80]	Hierarchical search space	Structural search for differentiable models based on delay metrics	Avoiding the process of exhaustive and independent training	Search on only a subset of the dataset	Lower latency than MNasNet-generated models with 420 times smaller search cost
ProxylessNAS [81]	Single-channel super network	Model parameter pruning	No need to use subset or proxy datasets	Inability to adapt to different hardware platforms	200x reduction in training time with higher accuracy and lower GPU latency
Single-Path NAS [83]	Hyperparameterized Super Networks Kernels with shared weights	Layer-by-layer search for kernel weights	Reduced search costs and number of model parameters	Restricted search space for single paths	Reduced search costs and number of model parameters
SPOS [82]	Single-channel super network	Uniform sampling and evolutionary algorithms	Decoupling network weights from network structure	Consistency constraints of super-networks in SPOS	Simplified super networks are easy to train and support complex search spaces
MnasNet [84]	Intensive learning	Intensive learning	Combining inference latency and accuracy	Large search space High search cost	Considering Mobile Reasoning Latency Constraints Exploring the importance of layer diversity
FNAS [85]	Performance Abstraction Model	Setting the reward function for pruning	No training required	Search quality depends on the reward function	Achieve 11.13x speedup of search process
EDD [86]	Fusion of deep neural networks Common search space for search variables and hardware-implemented variables	Gradient descent based optimization algorithm	Effective fusion of hardware and model parameters	Hardware-dependent implementation of accurate estimates of performance	Ability to search for models with comparable accuracy but better performance than the best available deep neural network in 12 GPUs/h
NASCaps [87]	Different types of algorithmic layer hyperparameter settings	Genetic NSGA-II based algorithm	First to simulate and support capsule layers	The number of configurations to explore is huge	New Deployment Ideas for High Accuracy Modeling at the Edge
APNAS [88]	Weight Sharing Circular Network Controller	Intensive learning	Search without synthesis or simulation	Search space is dependent on the accuracy of analytical models	Requires only 0.55 GPUs per day Provides 53% fewer cycles on average
HotNAS [89]	Based on a set of existing pre-trained models	Model Compression	Enables hot start and avoids long training sessions	Only two hardware design spaces were considered	Reduced search time from 200 GPU/h to less than 3 GPU/h
HURRICANE [90]	A variety of different convolutional and pooling layers	Two-stage search algorithm	Hardware diversity was considered	Restricted search space for model structure	2.59% to 4.03% improvement in accuracy on multiple platforms compared to MobileNetV2
[91]	The network architecture and hardware scheme comprising the pair of	Intensive learning	High flexibility and design freedom	Requires some search time	Better trade-off between test accuracy and hardware efficiency
[92]	Dynamic search spaces generated by black boxes	Selection algorithm based on cost function	No a priori knowledge of the underlying hardware is required	Only adapted to ISA-based gas pedals	Improvements in search time and GPU memory

HLS approach can effectively optimize the computation of complex functions and enhance design efficiency.

3. Hybrid Method Design: Some researchers have proposed hybrid design methods that merge the high performance of the RTL approach with the flexibility of the HLS method. This method typically employs HLS technology for coarse-grained design of model structures, while specific kernels and modules are designed using RTL methods for finer granularity. The hybrid method combines the advantages of both design strategies, ensuring high performance while increasing development flexibility.

Each design method has its unique advantages and application scenarios. RTL-based designs, while offering the highest performance, come with long development cycles and high complexity; HLS-based designs simplify the development process and improve design efficiency, but may require compromises on performance; hybrid method designs aim to find a balance between performance and flexibility; designs based on high-level frameworks further reduce development difficulty and accelerate the deployment of Neural Network Model on FPGAs.

3.3.1. Register Transfer Level (RTL)

RTL, short for "Register Transfer Level", represents a design abstraction level that describes digital circuits. RTL descriptions focus on data transfers between registers and the changes within registers, defining the logical structure and behavior of hardware operations. In RTL design, hardware description languages (such as VHDL or Verilog) are used to precisely describe the logic and timing behavior of digital circuits. The RTL method allows for manual optimization of specific hardware structures, enabling high-performance custom designs. This approach generally requires in-depth hardware design knowledge and significant time investment but can achieve very high performance and efficiency.

Based on different design granularities, RTL-based methods can be mainly categorized into the following three types:

Table 12
Comparison of advanced frameworks.

Tools/framework	Language/Platform	Characterization	Application/Optimization	Reference
Caffe	C++ Python MATLAB	Classical deep learning framework with support for generalized neural network models	Xilinx ML Suite recommended for hardware solutions on FPGAs	[93]
Caffe-OpenCL	OpenCL	Converting the Caffe framework to an OpenCL backend for enhanced cross-platform portability		
FixCaffe	C++	Replacing High-Precision Floating-Point Matrix Multiplication Functions with Low-Precision Fixed-Point Matrix Multiplication Functions	Performance optimization	[94]
Vitis	C C++ OpenCL	Xilinx's Unified Software Platform, Optimized Al Hardware Library	Simplified Hardware Design Flow for Xilinx FPGAs	
Quartus Prime	C++	Complete Development Toolkit for Intel FPGAs	Covering design synthesis optimization verification and simulation	
PyHDL	Python	C++ Object-Oriented Hardware Design Providing Scripting Interface	FPGA circuit generation using PamDC and PAM-Blox libraries	[95]
PHDL	Python	New hardware design language for building component libraries based on Python	Hardware components are selected based on input/output widths and target platforms	[101]
PyMTL	Python	Solving the problem of long simulation times by hybrid T-compilation and specialization methods	Accelerating the simulation process	[97]
PyRTL	Python	A way of describing the structure of the digital hardware, providing a wrapper around the "core" primitives.	Optimizing the digital hardware design flow	[98]
JHDL	Java	Hardware Description Language for Reconfigurable Systems to Manage FPGA Resources in an Object-Oriented Language	Analog/execution environment switching to manage FPGA resources	[99]
rHDL	Ruby	Ruby-based hardware description language	Manages FPGA resources and provides analog/execution environment switching	[100]

1. Template-Based Design: This method uses predefined hardware module templates, such as processing units or data paths for specific functionalities. These templates are optimized and can be applied across various designs. Suitable templates can be selected and configured to meet the requirements of specific applications, providing better performance than fully automated methods while reducing design complexity.

2. IP Core-Based Design: IP cores are pre-designed and verified hardware modules that can be invoked like functions in a software library. These cores can be generic, such as standard processor cores or specific communication interfaces, or optimized for particular computational tasks. Using IP cores allows for the rapid construction of complex systems while ensuring that each component is optimized and verified.

3. Custom Core-Based Design: When standard templates and IP cores do not meet specific performance requirements, designers opt for custom core design. This involves creating hardware descriptions from scratch, optimizing every part of the hardware entirely according to the specific needs of the application. This method typically offers the highest performance but is the most time-consuming and requires extensive expertise.

When designing FPGA accelerators, balancing the high performance of manual design with the efficiency of automated design is a key consideration. RTL-based methods provide designers with multiple choices, from highly customized to relatively automated options, allowing them to make appropriate decisions based on the specific needs of the project, time, and resource constraints (see Table 13).

3.3.2. High-Level Synthesis (HLS)

In the research on neural network optimization and acceleration methods for FPGAs, High-Level Synthesis (HLS) technology plays a significant role as one of the primary techniques in automated design tools. HLS tools enable developers to describe the behavior of hardware accelerators using high-level programming languages, such as C/C++ or OpenCL, and automatically generate HDL code. This approach greatly reduces development complexity and enhances design efficiency.

Template-Based Methods automate the mapping of algorithms to hardware through predefined computational patterns and architectural frameworks. Research [77] explores the automatic design for model-to-hardware deployment using stacked systolic arrays, optimizing computational performance and highlighting the advantages of the template method in standardized description and performance enhancement. Meanwhile, study [110] focuses on model pruning by trimming weights or even neurons to reduce data and computation volume, further enhancing computational efficiency. Studies [111,112] design unified templates based on the Winograd algorithm and matrix multiplication for 2D and 3D CNNs, respectively, showing the deep consideration of algorithmic features in template design.

IP Core-Based Methods generate hardware descriptions directly through automated design tools for parallel computation and data optimization. FINN [113] customizes computing resource configurations for convolutional and pooling layers, fpgaConvNet [114] optimizes hardware design using the Synchronous Data Flow (SDF) model, and Caffeine [68] achieves computation and communication binding layer optimization through a unified convolutional matrix multiplication representation, demonstrating the advantages of IP core methods in generalized design and flexibility.

Custom Core-Based Methods combine the high performance of manual design with the efficiency of automated design. CascadeCNN [115] optimizes resource consumption and computational precision by designing different precision control units, showcasing the advantages

Table 13
Comparison of RTL-based hardware design methods.

Method name	Design category	Model-specific	Realization details	Dominance	Inferior
[102]	RTL templates	Convolutional network	Dual cache	Low resource consumption	For handwritten digit recognition
[103]	RTL templates	Convolutional tri-Valued network	Variable bit width	Large performance trade-offs	Weighting and activation using only three-valued values
DeepBurning [104]	RTL level generator	Neural network	Co-compiler	Coverage of multiple design processes	Lack of optimization space for algorithmic dimensions
Auto deep neural network [105]	IP selection; Can synthesize RTL code	Deep neural network	Image gas pedal representation	High degree of automation	Optimization space is limited
CaFPGA [106]	Hardware design flowchart; IP libraries	Convolutional network	Layer Folding Pipe Structure	Balancing bandwidth requirements betweentiers	FPGA implementation for CNNs only
Deep Neural Networks Weaver [107]	Macro Coarse Grain ISA Modular Templates	Deep neural network	Model slice	Leverage reconfigurability	Slicing operations consume additional design costs
PLACID [108]	FCE Computing Units; RTL-level architecture	Convolutional network	Towards spatial characterization and exploration	Automated Optimization of Convolutional Operations	Only for convolutional network models
Deep Neural Networks Builder [109]	RTL neural network components	Deep neural network	Layer pipeline; Column caching scheme	Higher resource utilization	Lack of optimization space for algorithmic dimensions

Table 14
Comparison of HLS-based hardware design methods.

Method name	Design category	Model-specific	Realization details	Dominance	Inferior
[111,112]	HLS templates based on generalized algorithms	2D and 3D convolutional networks	Matrix multiplication expression	Low design costs	Constrained optimization space
[77]	Pulsed array based HLS templates	Convolutional network	Pulsed Array Architecture	High resource utilization	Hardware solutions depend on algorithmic mapping quality
[110]	HLS Templates	Deep neural network	Weight pruning	Reduced storage pressure	Only for pruned network models
Caffeine [68]	Caffe-based hardware/software co-design library	Convolutional Networks Fully connected network	Convolutional matrix multiplication representation	Implementation of Unified Convolution and Matrix Multiplication	Lack of performance comparison with other methods
fpgaConvNet [114]	Reconfigurable Hardware Networking Framework	Convolutional network	Simultaneous data stream computation model	Multi-objective optimization is possible	Only for convolutional models
Finn [113]	HLS Hardware Library	Binarized network	Heterogeneous Streaming Architecture	Customizable resources per layer	Only for binarized networks
Angel-Eye [118]	Programmable computing units	Convolutional network	Data quantification	Responds to network topology changes	Lack of other convolutional network model validation
CascadeCNN [115]	Multiple Precision Computing Units	Convolutional network	Multi-Resolution Convolutional Modeling	High model accuracy	Face detection only
[116]	PCE module	Convolutional network	Layer parallelism; Kernel parallelism	Effective use of parallelism sources	Inference phase for convolutional networks only
[117]	Highly configurable HLS IP	Long-term recurrent convolutional networks	Network pruning; Weight retraining	Low power consumption and latency	Only for long term recurrent convolutional networks

of custom core methods in detailed design and performance optimization. Additionally, research [116,117] delve into parallel computational units and data access optimization, while Angel-Eye [118] enhances the generality of hardware design through quantization strategies and programmable computing engines.

Optimization and acceleration methods for neural networks on FPGAs cover comprehensive strategies from template design and IP core generation to custom core development, aiming to achieve higher computational efficiency, lower resource consumption, and better performance outcomes (see Table 14).

3.3.3. Hybrid approach

In the research on accelerating Neural Network Model on FPGA platforms, the choice of hardware-level design methods is crucial for optimizing model performance and accelerating the design process. Manual hardware-level design (RTL) methods allow for efficient and significant acceleration, ensuring fine control and high optimization. In contrast, High-Level Synthesis (HLS) methods enhance design efficiency by speeding up the transformation from algorithms to hardware. Hybrid methods combine the high-performance advantages of RTL design with the rapid development capability of HLS methods, aiming for a system-level integrated design that leverages the strengths of both approaches.

However, hybrid methods in practice require managing both RTL and HLS design elements, increasing design complexity and integration difficulty. To address this challenge, some research works have explored innovative integration solutions. For example, Field-Programmable Deep Neural Networks (FP-DNNs) [119] adopted an

Table 15
Comparison of Hybrid Hardware Design Methods.

Method name	Design Category	Model-specific	Realization details	Dominance	Inferior
ALAMO [120]	HLS Compiler RTL Module	Convolutional neural network	Parameterized RTL modules	Highly scalable	Higher design complexity
HeteroCL [121]	Domain-specific language	Neural network	Programming abstraction	Adaptation to different workloads	Increased design complexity
[124]	HLS Compiler RTL Module	Deep neural network	Computational primitive language integration (CPI)	Modular; highly scalable	Limited scalability
[122]	HLS Compiler RTL Module	Convolutional neural network	Hierarchy	Suitable for different layers of operation	Increased design complexity
FP-Deep Neural Networks [125]	RTL-HLS hybrid templates	Network inference	Operating category	Accelerated reasoning	Requires model preprocessing
[123]	RTL-HLS hybrid templates	Convolutional neural network	Compression; Decompression	Reduced data bandwidth requirements	Increased computational complexity

RTL-HLS hybrid template automation strategy to generate FPGA hardware implementations, accelerating the model inference process. Additionally, ALAMO [120] introduced a compiler that integrates HLS and RTL, capable of converting CNN models generated by frameworks like Caffe or Theano into parametrized RTL modules, further enhancing design flexibility and applicability.

In terms of integration dimensions, the HeteroCL [121]framework conducts in-depth exploration of computational dimensions by separating and analyzing the relationship between algorithm specifications and computation, data types, and memory architectures. This framework generates hardware implementations for spatial architecture templates, such as systolic arrays, providing effective solutions for processing various popular workloads. On the other hand, research [122] optimized the model structure dimension by abstracting the model structure into a Directed Acyclic Graph (DAG), achieving reconfiguration and consistency with RTL modules during compilation. Regarding the data dimension, research [123] explored strategies for designing trade-offs between computational logic and data bandwidth by applying data compression techniques, optimizing storage and transmission efficiency. These explorations of integrated methods and dimensions not only showcase the diversity and complexity of implementing deep learning model acceleration on FPGA platforms but also highlight potential pathways for optimizing the design process and improving model performance (see Table 15).

4. Algorithm hardware collaborative optimization method

4.1. Algorithm-hardware co-design

Within the framework of general-purpose computing, the design of computer systems adopts a layered abstraction approach for software and hardware, allowing for independent development of software and hardware design while ensuring the system's universality and flexibility. In such architectures, software algorithms are typically developed at the level of high-level programming languages, without the need to delve into the underlying hardware structure. Similarly, the design of hardware processors mainly targets general instruction sets, without specifically optimizing for the characteristics of upper-layer software algorithms. This level of division not only simplifies the design process but also enables interlayer collaboration through well-defined interfaces. The abstraction between layers also results in performance overhead, such as the performance difference between programming in high-level languages and assembly language on general-purpose processors, as well as the performance difference between general-purpose processors and custom accelerators, all of which directly reflect this design methodology (see Fig. 4).

4.1.1. Mismatch between algorithms and hardware

The design of algorithms and the design of hardware exhibit an inherent contradiction, primarily manifested in their different demands for computation and memory access characteristics. The goal of algorithms is to solve application-level problems, enhancing problem-solving capabilities and application efficiency, while the objective of hardware is to achieve high-speed and efficient execution. Algorithms tend to favor unrestricted computation and memory access characteristics to handle more complex and diverse problems, often leading to a contradiction between the computational and memory access characteristics exhibited by algorithms and those required for efficient hardware execution. Hardware's computational and storage resources are limited, necessitating effective and efficient utilization of these resources. Therefore, hardware design tends to favor algorithms with fewer computations and memory accesses, more balanced ratios, and more regular patterns.

With technological advancements, the hardware side has alleviated this contradiction by enhancing computational capabilities and optimizing microarchitectures to support efficient algorithm execution. However, due to the stagnation of Moore's Law and Dennard scaling, improvements in hardware performance face bottlenecks, especially when dealing with irregular and unordered computation and memory access patterns. The existing optimizations in general processor microarchitectures or dedicated hardware customization strategies still fail to address issues such as computational load imbalance, waiting for memory access during computation, memory access conflicts, and cache thrashing, all of which limit hardware efficiency in processing irregular and unordered patterns. In this context, simply optimizing from either the hardware or software perspective alone is no longer sufficient to meet the growing computational demands. The co-design of algorithms and hardware becomes an important solution, considering the characteristics of the hardware platform during the algorithm design phase and optimizing the algorithm, as well as customizing computation and storage structures according to algorithmic features during the hardware design phase, thereby achieving an overall performance improvement.

4.1.2. Algorithm-hardware co-design method

The computation and storage model acts as a pivotal bridge between algorithm design and hardware design, facilitating the evolution of a co-design methodology. This approach is founded on the principle of optimizing algorithms to align with the computational and storage characteristics of hardware and customizing hardware to expedite the execution of specific algorithms, aiming for the objective of achieving high-performance computing. Characteristics conducive to hardware include reducing the number of operations, balancing the proportion of operations, and regularizing patterns. Consequently, this co-design strategy involves two complementary aspects: hardware-oriented algorithm optimization and hardware customization based on algorithmic features.

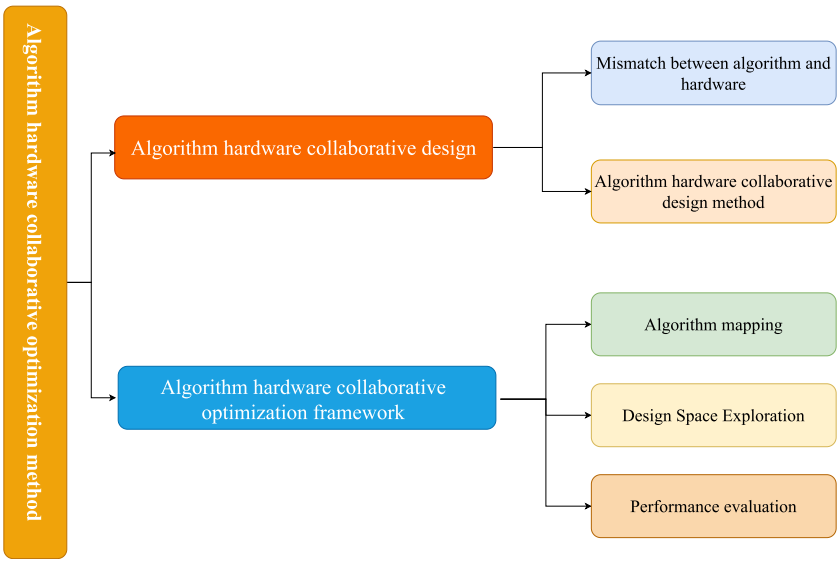


Fig. 4. Design framework diagram of algorithm hardware collaborative optimization method.

Algorithm optimization seeks to decrease the number of computational and storage operations through methods like data compression and algorithm or model pruning, as well as by eliminating irregular computational or storage operations to regularize operation patterns, thereby making computational and storage operations more amenable to hardware execution. On the other hand, hardware customization focuses on microarchitecture optimization, storage hierarchy customization, and leveraging parallel granularity according to the features of the algorithm, with the goals of reducing latency, diminishing energy consumption, and enhancing processing throughput.

In the process of algorithm-hardware co-design, it is essential to balance the design objectives of algorithm effectiveness and hardware efficiency. Algorithm design concentrates on upper-layer applications, such as the performance of artificial intelligence applications, which includes, but is not limited to, accuracy, error rates, and recall rates, while also considering security and robustness. Hardware design focuses on the underlying processors or custom hardware, aiming to efficiently utilize computational and storage resources, with efficiency metrics including latency, throughput, power consumption, as well as considerations of development cycles and costs.

Implementations of Deep Neural Networks (DNNs) on FPGA exemplify the advanced methodology of software-hardware co-optimization. FPGAs provide a flexible hardware platform that allows for the customization of hardware logic to meet the specific needs of algorithms, thereby facilitating efficient execution of DNNs. In this context, software-hardware co-optimization involves not only adjustments and optimizations at the algorithmic level to adapt to hardware characteristics and enhance execution efficiency but also includes hardware-level custom designs to maximize algorithm performance. Here are some aspects of software-hardware co-optimization in the implementation of DNNs on FPGAs:

1. Algorithmic-Level Optimization: At the algorithmic level, strategies such as network pruning, quantization, and low-rank decomposition are employed to reduce computational complexity and storage requirements. These optimizations diminish the size and computational burden of models, rendering Neural Network Model more amenable to execution on resource-limited FPGAs. Moreover, by tailoring algorithm design to leverage FPGA's parallel processing capabilities, network structures can be adjusted to optimize the utilization of hardware resources.

2. Customized Hardware Design: The programmability of FPGAs allows for hardware-level optimizations tailored to specific deep neural network models or algorithmic features. This includes customized data path designs, dedicated accelerator implementations, and storage architecture optimizations that utilize local storage and pipelining techniques to minimize memory access latency and enhance throughput. Hardware customization facilitates the creation of optimal hardware architectures that cater to specific algorithmic requirements, such as parallelism and data reuse strategies.

3. Resource and Power Management: On FPGAs, the management of logic units, storage resources, and power consumption are critical metrics for optimization. Software-hardware co-optimization seeks a balance between algorithmic efficiency and hardware resource utilization, combining algorithm adjustments and hardware design to achieve efficient resource use and power optimization.

4. Dynamic Reconfiguration Capability: The dynamic reconfiguration capability of FPGAs offers additional possibilities for software-hardware co-optimization. Depending on runtime requirements, hardware configurations on the FPGA can be dynamically adjusted to support various algorithms or models, thereby increasing hardware flexibility and resource utilization.

5. System-Level Optimization: Beyond optimizing individual deep neural network models, software-hardware co-optimization can be conducted at the system level. By optimizing data flows and processing sequences, data transmission delays can be reduced, enhancing the overall efficiency of the system execution.

4.2. Algorithm-hardware co-optimization framework

In FPGA neural network optimization research, the software-hardware co-design framework can be divided into three key steps, which together ensure the efficiency and practicality of the design. These three steps are:-Algorithm Mapping,Design Space Exploration (DSE),Performance and Resource Consumption Evaluation (see Table 16 and Fig. 5).

4.2.1. Algorithm mapping

Algorithm mapping is a critical step in generalized design schemes, as it involves transforming high-level algorithms into specific implementations on low-level hardware platforms. The core objective of algorithm mapping is to ensure that algorithms can efficiently run on hardware and meet system performance requirements. Firstly, by analyzing the computational characteristics of algorithms, algorithm mapping optimizes the utilization of hardware resources, ensuring that the system achieves the desired computational performance. Secondly, the diversity and flexibility of neural network models require

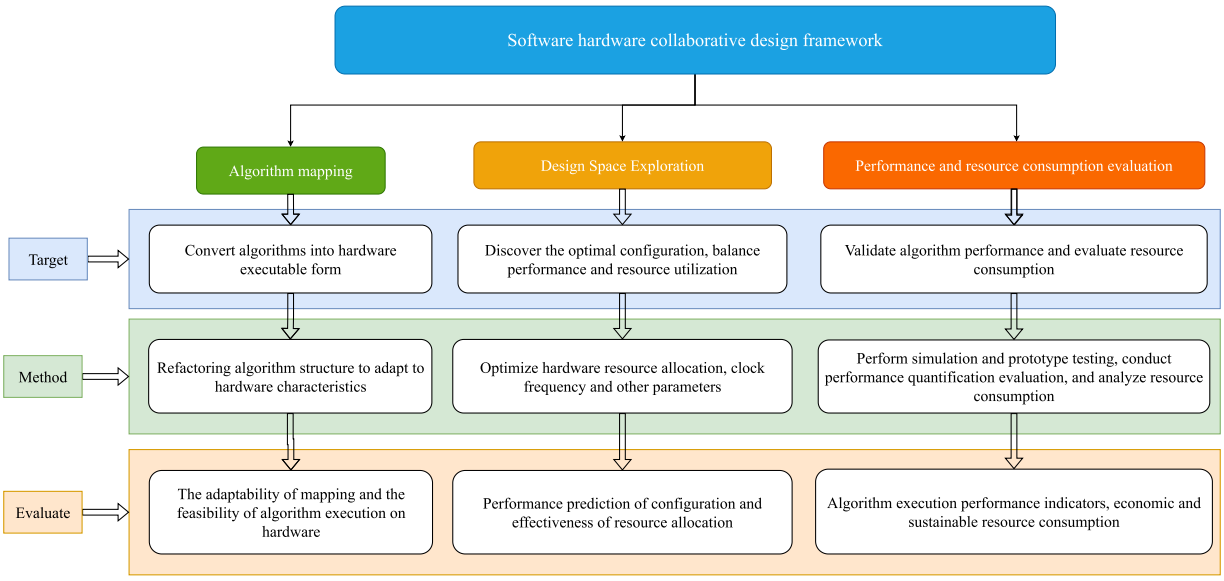


Fig. 5. Algorithm hardware collaborative optimization framework diagram.

Table 16
Steps in Algorithmic Hardware Collaboration.

Step	Objectives	Methodology	Evaluation
Mapping of algorithms	Translate algorithms into hardware executable form	Reconfiguration of algorithm structure to adapt to hardware characteristics (e.g.,fixed-point arithmetic, parallel processing)	Adaptation of mappings and execution possibilities of algorithms on hardware
Design Space Exploration	Discovering optimal configurations balancing performance and resource usage	Optimize hardware resource allocation clock frequency, and other parameters	Configuration performance prediction resource allocation effectiveness
Performance and Resource Consumption Evaluation	Verify algorithm performance and evaluate resource consumption	Execute simulations prototype tests quantitative performance evaluation resource consumption analysis	Algorithm execution performance metrics economy and sustainability of resource consumption

that algorithms be adaptable to various hardware platforms. By defining common computational characteristics, algorithm mapping enables compatibility of different model algorithms within a unified computational framework. Additionally, algorithm mapping focuses on resource optimization, particularly in handling sparse data, where it effectively reduces redundant computations and data movements, thereby enhancing overall efficiency. Lastly, algorithm mapping serves as the foundation for design space exploration; only after successfully mapping the algorithm onto hardware can further exploration and optimization of design parameters occur to identify the optimal design solution. Therefore, algorithm mapping plays a crucial role in bridging high-level algorithm design and low-level hardware implementation, ensuring the fulfillment of performance requirements and providing a solid foundation for efficient hardware resource utilization and design space optimization.

In FPGA-based neural network optimization, algorithm mapping typically revolves around two directions: customization and generalization. The choice of direction depends on the required performance, resource efficiency, and specific needs of the network model.

1. Customized algorithm mapping

In the field of FPGA-based neural network optimization, customized algorithm mapping strategies occupy a central position, especially in applications seeking ultimate computational performance and high resource efficiency. This strategy achieves hardware design that precisely matches model requirements by tailoring hardware logic for specific Neural Network Model or computational tasks. The core advantage of customized mapping lies in its ability to maximize the use of FPGA's parallel processing capabilities and flexible configurability, accommodating the unique demands of various computationally intensive models from complex convolutional neural networks to graph neural networks.

The technical implementation of customized algorithm mapping involves multiple layers of optimization and innovation. In terms of hardware design, the customization mapping strategy may involve designing highly parallel processing units for specific deep learning operations, such as convolution, pooling, or activation functions. These processing units are meticulously designed to maximize processing speed while minimizing resource consumption. Additionally, to support complex data flows and efficient memory access, customized mapping requires developing complex data flow control logic and memory access strategies, utilizing techniques for data reuse, prefetching, and cache optimization. Customized mapping strategies not only improve computational efficiency but also significantly reduce memory access latency, thereby enhancing the execution speed and efficiency of neural network models overall.

Classic optimization methods such as GEMM and the Winograd algorithm leverage the parallel processing capabilities and efficient memory access strategies of FPGAs to minimize memory access in matrix operations and reduce the number of multiplications required for convolution operations. The GEMM (General Matrix Multiplication) method vectorizes multiplication operations with the same weights, thereby minimizing memory access during matrix multiplication. This method is particularly efficient when handling large matrices, especially in scenarios involving fully connected layers in neural network models. Experimental data show that implementing GEMM on FPGAs can achieve up to 20 times the computational performance of CPUs while reducing memory bandwidth usage by 50% [126].Several OpenCL-based GEMM frameworks, such as Intel's FPGA SDK for OpenCL, have been

Table 17
Comparison of customized algorithmic mapping methods.

Optimization Strategy	Application Scenarios	Performance Improvement Effect	Reference
GEMM	Fully connected layer	Minimizing memory accesses to optimize large matrix computations	[130]
Winograd	Convolutional computation	Reduces the number of multiplication operations increasing bandwidth and throughput	[131,132]
Algorithm mapping to kernel	Sparse Convolutional Networks	Customization for models or algorithms Reduced resource utilization	[133–135]
PRA	Bit pruning	Reduces memory access overhead and skips invalid computations	[136]
SCNN	Zero-weight pruning	Reduce computation by pruning with zero-valued weight sums	[137]
O3BNN-R	Edge Evaluation Optimization	Enhanced loss function to reduce runtime edge evaluation	[138]
SnaPEA	Activation Prediction and Skip Execution	Improve efficiency by predicting activations in advance to skip execution	[139,140]

widely adopted to optimize matrix multiplication operations on FPGA platforms, further enhancing computational efficiency and resource utilization.[127]

The Winograd algorithm is widely applied in convolutional computations within convolutional neural networks. It leverages the principles of fast Fourier transforms to transform convolution operations in the time domain into pointwise multiplications in the frequency domain, thereby reducing the number of multiplications required. According to experimental results, the use of the Winograd algorithm for convolution operations in the ResNet-50 model achieved a 2x acceleration in computation and reduced computational resource usage by 30% [128]. However, despite significantly reducing computational load, the Winograd algorithm imposes higher demands on system bandwidth and throughput. In some applications, this algorithm has been integrated into several network model inference frameworks, such as TensorFlow and Caffe, to further optimize the efficiency of convolution operations [129]. These findings indicate that GEMM and the Winograd algorithm provide effective computational optimizations on FPGA platforms, significantly enhancing the inference performance of Neural Network Model while reducing computational resource consumption.

The direct mapping of algorithms to kernel strategies is special, including mapping software optimizations directly to FPGA hardware kernels, allowing software optimizations to realize the customized features of models or algorithms, while hardware kernels remain general. This approach effectively utilizes hardware resources and algorithm characteristics, adapting to different neural network structures and sizes, and designing computational cores for resource-constrained devices.

Computational pruning strategies can be divided into fine-grained bit pruning and coarse-grained computational pruning. Bit pruning reduces computation and storage needs by ignoring bits that do not participate in effective operations, such as the PRA architecture, which skips zero-bit inputs through serial parallel shift and addition. Coarse-grained pruning utilizes zero-value weights for pruning, like Sparse CNN (SCNN) that leverages zero values in weights to reduce computational demands.Regularization and early prediction optimization methods are applied during the training phase, optimizing the network structure by introducing specialized regularization terms to enable early determination of neuron outputs during inference, thus reducing edge evaluation. This method skips certain computations by predicting activations early, reducing unnecessary operations without sacrificing overall system efficiency (see Table 17).

2. Generic algorithm mapping

Generalized algorithm mapping in FPGA-based deep learning optimization aims to provide a flexible and scalable hardware design framework, involving everything from modular design to advanced programming techniques. This approach seeks to maximize universality and flexibility while minimizing the impact on performance to support a wide range of Neural Network Model and computational tasks.

The classification of generalized algorithm mapping includes three main categories: Modular Design: Modularization involves decomposing hardware design into reusable modules, each performing specific operations or computational tasks. The core of modular design is creating a series of standardized, reusable hardware modules, such as implementations of convolution, pooling, fully connected operations for deep learning. Each module is designed to execute one or a set of specific operations, with uniform interfaces, allowing them to be flexibly combined to fit different model architectures. Layer-Wise Design, a modular approach, transforms each layer of a deep learning model into an independent hardware module. These modules are organized on FPGAs in a pipelined or parallel fashion to support continuous data processing. Modular design allows for quick replacement or updating of certain layers in a model without the need to redesign the entire hardware architecture.

Configurable Computing Resources: This category of algorithm mapping focuses on developing computing units that can dynamically adjust their structure and parameters based on model requirements. These computing units can be general-purpose processor cores or accelerators optimized for specific computational tasks, such as convolution computations, also referred to as general-purpose accelerators. Dynamic Precision Scaling technology enables dynamic adjustment of the data bit-width of computing resources based on runtime precision requirements. For simple tasks or certain parts of a model, lower precision can be used to reduce computational and storage requirements, while for computations requiring high precision, the bit-width can be dynamically increased.

Data Flow Optimization: Data flow optimization focuses on efficiently managing and scheduling the flow of data between hardware modules. This includes designing efficient data paths, caching mechanisms, and prefetch strategies to reduce memory access latency and improve computational throughput. Smart Cache Systems utilize the principle of locality to intelligently cache data that will soon be used, reducing access to slower storage (such as external DRAM). By analyzing the data access patterns of a model, prefetch strategies can preload data into faster internal storage before it is processed (see Table 18).

The advantage of generalized algorithm mapping lies in its ability to maintain flexibility and universality. However, generalized algorithm mapping may not perform deep optimization for specific models, which could result in suboptimal performance in some performance-intensive application scenarios. For instance, for convolution operations requiring high optimization, generalized mapping might not be as efficient as custom hardware specifically designed for that operation. While generalized mapping allows hardware to support a variety of computational tasks, this versatility may lead to underutilization of hardware resources (such as logic units and storage) in certain

Table 18
Comparison of generalized algorithmic mapping methods.

Classification	Specific Methods	Core Ideas	Technical Examples
Modular Design	Layer-Wise Design	Create a series of standardized, reusable hardware modules such as convolution, pooling, full connectivity, etc.	Transform each layer into independent hardware modules that support pipelined or parallel processing.
Configurable Computing Resources	Generic Accelerator + Dynamic PrecisionScaling	Develop computational units that can dynamically adjust structure and parameters according to model requirements.	Dynamically adjust data bit widths according to accuracy requirements.
Data Flow Optimization	Smart Cache Systems	Efficiently manages the flow of data between hardware modules and reduces memory access latency.	Utilizes the principle of locality to intelligently cache data and implement prefetching strategies

specific tasks, thereby affecting overall resource utilization. Addressing the challenges of designing a hardware platform that maintains high universality while meeting the needs of different Neural Network Model involves addressing numerous design challenges, including data flow management, memory access optimization, and computational resource configuration. This increases design complexity and development difficulty.

While providing FPGA with flexibility and broad applicability, generalized algorithm mapping also faces challenges of performance compromise and low resource utilization. A balance between universality and performance must be struck, through careful design and optimization, to achieve effective deployment across varied application scenarios.

4.2.2. Design Space Exploration

Design Space Exploration (DSE) is a pivotal process in the realm of hardware acceleration, particularly in the acceleration of Neural Network Model on FPGA platforms. DSE entails systematically searching and evaluating different hardware configurations and architectural choices to identify the hardware design most suited to specific application requirements. This process involves a broad spectrum of hardware parameters within the context of FPGA acceleration, providing a framework to find a balance among multiple design objectives, such as performance, resource consumption, and power efficiency.

Specific research directions include exploring the trade-offs and interactions between different design choices and developing new strategies that can effectively extend the utilization rate of the design space. Through such exploration, researchers can identify which configurations and architectural choices can maximize model performance under given algorithmic and hardware constraints, considering factors like the allocation of computational resources, optimization of storage capacity, configuration of data pathways, and organization of processing units, with the aim of achieving optimal collaboration between algorithms and hardware design.

When undertaking DSE, researchers must conduct an in-depth analysis of potential design choices to locate and expand the feasible design space. This requires not only a thorough understanding of existing design choices but also the innovative development of new methods to more efficiently utilize the design space, especially in achieving high performance on FPGA platforms. Literature discussions indicate that effective design space exploration can not only optimize model performance on FPGAs but also provide crucial guidance for hardware design, fostering more efficient and flexible deployments of Neural Network Model. Although the DSE process itself can be complex and time-consuming, it is indispensable for realizing efficient and scalable FPGA deep learning acceleration solutions.

High-Level Synthesis (HLS) technology offers effective tools to tackle this challenge. HLS allows designers to work at a higher level of abstraction, using high-level programming languages such as C/C++ or SystemC, and then automatically translates these designs into lower-level implementations, such as Register Transfer Level (RTL) or gate-level netlists. This process not only simplifies hardware design and verification efforts but also significantly accelerates the development cycle. Over the years, HLS technology and its tools have been extensively researched and applied in both academic and industrial contexts.

Modern HLS tools provide a wealth of loop optimization options, such as loop unrolling, pipelining, array partitioning, and function inlining, the combinations of which grow exponentially, presenting opportunities and challenges for effective design space exploration. Dealing with timing constraints during the design process, the reuse of FPGA CAD tools is crucial for timing convergence. However, lengthy runtimes and complex configuration options significantly increase the difficulty of design process management for developers. Therefore, developing effective design space exploration strategies becomes a key research task in the study of FPGA-based deep learning model acceleration technologies. This requires developers not only to be proficient in hardware design principles but also to have a deep understanding of HLS tools and optimization strategies.

DSE is a systematic search process that allows designers to evaluate and select the most suitable hardware configurations for specific application scenarios. Given that design spaces often have a multi-objective nature, DSE frequently results in a set of optimal solutions, known as Pareto-optimal solutions, rather than a single optimal solution. These Pareto-optimal solutions offer the best trade-offs among different design objectives, but further validation and testing are needed to determine which solutions are more effective in practical applications. Efficient DSE techniques not only alleviate the pressure during the design process but also automate complex verification and testing workflows, quickly identifying a range of optimal solutions that meet specified criteria.

DSE can be divided into three key components: determination of evaluation criteria, construction of design models, and application of optimization strategies. First, the determination of evaluation criteria is a crucial step that clarifies the main factors for assessing the quality of different design solutions, such as system performance, energy consumption, resource utilization, and economic cost. Accurate evaluation criteria guide the entire DSE process and serve as the foundation for identifying Pareto-optimal solutions. Second, the construction of design models is critical to ensure that models can accurately predict the impact of different configurations on the evaluation criteria. Methods for building these models may include theoretical analysis, simulation, and even advanced techniques like machine learning, with the goal of enhancing model prediction accuracy and search efficiency. Finally, the application of optimization strategies involves using appropriate algorithms to systematically search the design space and identify the optimal set of solutions on the Pareto frontier. These optimization strategies can be based on traditional mathematical techniques or modern heuristic algorithms, such as genetic algorithms and particle swarm optimization. These methods excel in handling complex multi-objective optimization problems, effectively locating optimal solutions within vast design spaces (see Table 19).

1. The effectiveness of Design Space Exploration (DSE) heavily relies on the selection of appropriate evaluation criteria. These criteria need to intuitively describe and predict the impact of different design solutions on the efficiency of the target network model while

Table 19
Comparison of Design Space Exploration Methods.

Method name	Selection of indicators	Model building	Optimization methods	Realization details	Dominance	Inferior
TyTra [141]	Performance/ resource consumption	Roof model	Linear analysis	Type Conversion; Lightweight cost mechanism	Type conversion and design variants available	For stream processing mode only
[142]	Performance/ resource consumption	Layered Roof Models	Linear analysis	CAD Tools; HLS optimization	Assistance with HLS optimization for C/C++ applications	Dependent on the accuracy of the roofline model
[143]	Performance/ resource consumption	IP core-performance correlation model	Linear analysis	IP Mapping; Shared Bus	Highly scalable	Higher design costs
[144]	Performance/ resource consumption	Roof model	Linear analysis	Layering; Cyclic unfolding	Combined roof model and DSE engine	Requires manual optimization for hardware platforms
MPSeeker [145]	Computing/ communication costs	Integrated tree model	Machine learning	Multi-Granularity Parallelism	High parallelism	Parallel optimization does not consider algorithmic features
Lin Analyzer [146]	Computing/ communication costs	Dynamic data dependency diagram	Linear analysis	LLVM; A Multi-Stage Exploration	Avoid time-consuming HLS runs	Explore the cumbersome process
COS-MOS [147]	Computing/ communication costs	Time signature chart	Linear analysis	Linear Programming; Component characterization	Effectively reduce the number of HLS calls	Only loosely coupled gas pedals are supported
Sherlock [148]	Composite indicators	Agent model	Machine learning	Multi-objective optimization; Active learning	Robust	Dependence on data tagging quality
[149]	Composite indicators	Machine learning model	Machine learning	Feature Extraction; Model training	Resolve discrepancies between advanced synthesis and actual operation	Reliance on a large number of high-level synthesis reports
[150]	Performance/ Power Consumption	Roof model	Transfer learning	Similarity metric	Effective use of prior design exploration results	Dependent on prior design to explore quality
[151]	Performance/ Area	Hybrid shared multidomain model	Transfer learning	Multidomain migratory learning	Reusable knowledge of different but related domains	Dependent on domain-related knowledge
[152]	Performance/ resource consumption	Roof model	Model order optimization methods	Gaussian processes; The HLS command	Associate multiple uses of the same instruction	Dependent on advanced synthesis commands
[153]	Performance/ Area	Clustering-based modeling	Heuristic algorithm	Iterative clustering; The HLS Directive	Highly scalable	Search quality depends on initial sample set size
COMBA [154] [155]	Performance/ resource consumption	Optimization instructions and resource sharing	Linear analysis	Metric bootstrapping; Array division	Supports multiple optimization commands	Cannot generate hardware implementations directly
ACDSE [156]	Performance/ resource consumption	Models based on adaptive compression	Intensive learning	Model compression; Pruning	Leveraging Model Compression for Multi-Granularity Exploration	Only for convolutional models
Prospector [157]	Performance/ resource consumption	Bayesian probability model	Machine learning	Iterative sampling; Probability distributions	Low computing resource requirements	Requires large number of iterations and suffers from local optimization problem
[158]	Computing/ communication costs	Periodic estimation model	Random forest	Variable loop boundary	Solving the hard-to-optimize variable loop boundary problem	Relies on the accuracy and completeness of HLS tools
IronMan [159]	Performance/ resource consumption	GNN-based performance prediction model	Intensive learning	Code Conversion; HLS Tools	Effectively help HLS tools generate programs	Dependence on HLS tools and user-specified constraints
[160]	Composite indicators	Clustering-based modeling	Heuristic algorithm	Lattice traversal; Probability distributions	Highly scalable Reduced HLS operation	A certain number of initial samples are needed to explore
ERDSE [161]	Performance/ resource consumption	A framework based on reinforcement learning	Intensive learning	Separation of sampling and learning stages	High sample utilization	Only for convolutional models

considering resource consumption on the target hardware platform. Performance metrics, as one of the most common choices, provide a practical assessment benchmark for a variety of DSE problems due to their simplicity and intuitiveness. Research efforts such as TyTra [141, 142,144], and [143] focus on comparing solutions using metrics of performance and resource consumption. The direct relevance of this approach makes it a suitable option for various DSE issues, though it may introduce modeling errors due to unclear trade-off relationships between metrics.

Computational metrics, another important set of evaluation criteria, aim to describe and predict the ratio of delays or power consumption between data computation and data transmission across different design solutions. This metric is particularly useful for identifying key bottlenecks in solutions and guiding optimization directions. The roofline model, which compares compute-intensive operations to memory access operations, is a classic example of using this metric. For instance, MPSeeker [145], Lin Analyzer [146], and COS-MOS [147] adopt computation/communication cost models, where explicit trade-off relationships help clarify subsequent exploration directions. However, the limitation of such metrics lies in the need to build specific models for different models and hardware platforms, and they are not easily applicable to other types of DSE problems.

In practice, selecting suitable evaluation criteria often requires customization based on specific issues and tasks or conducting a comprehensive analysis using multiple metrics. For example, Sherlock [148, 150] provide a more flexible and comprehensive solution evaluation framework by selecting the most appropriate evaluation criteria from multiple proxy models based on the target task.

2. Constructing efficient design models is a key step in optimizing hardware configurations. The Roofline model, widely utilized in the construction of design models, is valued for its simplicity and intuitive nature, offering a means to examine all potential design approaches and their applicability across diverse design spaces. This model guides design space exploration by establishing upper limits for computational performance and memory bandwidth, allowing designers to intuitively identify performance bottlenecks.

Despite the significant advantages of the Roofline model in terms of flexibility and scalability within design space exploration, it also faces challenges related to high computational complexity and a lack of consideration for specific model and hardware platform characteristics. This is primarily because the Roofline model requires comprehensive computation and assessment of all possible solutions within the design space, potentially leading to substantial computational burdens in practice. To address this issue, some research works have proposed improvement strategies. For instance, TyTra [141,150], and [152]directly employ the Roofline model as a foundation for exploring the hardware design space, while [142,144] introduce a hierarchical Roofline improvement model aimed at mitigating the challenges posed by extensive design spaces. These improved models reduce computational complexity and enhance adaptability to specific hardware platforms and algorithmic characteristics through the introduction of hierarchical strategies.

Although the Roofline model, as a widely used analytical tool, is very effective in certain scenarios, it does not encompass all complexities of design decisions. Hence, the research community has developed various other models to provide more detailed performance-cost analyses and optimization guidance for specific scenarios.

MPSeeker [145] proposes a model similar to the Roofline but offers a more detailed analysis by defining performance costs as the sum of kernel computation and data communication, providing a more concrete perspective for analysis and optimization. This approach is particularly suited to scenarios of accelerating Neural Network Model, where balancing computation-intensive and data-intensive tasks is crucial for overall performance.

Furthermore, researchers have designed a series of specialized models catering to the specific characteristics of different models and hardware platforms. For example, the Lin Analyzer [146], based on data dependency graphs, focuses on analyzing data flows and task dependencies to guide optimization of data transmission and computational tasks. The clustering-based approach [153] reduces the size of the search space by clustering similar design solutions within the design space, enhancing the efficiency of DSE. Research based on existing solutions [151] quickly locates potential optimization spaces and identifies performance bottlenecks by analyzing and comparing existing design approaches.

3. In the realm of Design Space Exploration (DSE) for hardware acceleration, particularly in accelerating Neural Network Model on FPGA platforms, optimization strategies can be categorized into linear analysis methods, machine learning approaches, and reinforcement learning or heuristic algorithm techniques.

Model-based linear analysis methods serve as traditional optimization strategies, boasting clear advantages in solid theoretical foundations and a transparent exploration process, typically yielding global optimum solutions. However, these methods are constrained by assumptions about the problem, potentially faltering in the face of complex design challenges, especially when confronting vast design spaces due to substantial computational resource requirements and extended computation times. To overcome these hurdles, High-Level Synthesis (HLS) methods are extensively utilized to simplify the complexity of DSE. By exploring across code, computing kernels, and computation dimensions, HLS methods offer an efficient framework for design space exploration. For instance, research works such as [142] and COMBA [154,155] utilize C/C++ source code input and leverage LLVM intermediate representation to track the design space, effectively tracing and optimizing different variables throughout the design process. Additionally, studies [143,144]explore potential optimization solutions by selecting different computing kernels, significantly enhancing model performance.

Further, Lin Analyzer [146], Lina [162], and COS-MOS [147] focus on choosing the best optimization strategy across different computation dimensions. This approach considers not only algorithmic-level optimizations but also delves into the specifics of hardware implementation, thereby providing more refined and efficient acceleration schemes for Neural Network Model on FPGA platforms.

While model-based linear analysis methods theoretically hold certain advantages, they may encounter limitations in practical applications. By integrating High-Level Synthesis (HLS) methods and comprehensively considering various aspects such as code, computing kernels, and computation dimensions, researchers can effectively simplify the process of design space exploration, swiftly identifying optimal design solutions.

Machine learning methods are primarily praised for their broad applicability, capability to handle complex problems, and high flexibility and adaptability. By learning from extensive data, these methods can accurately capture the characteristics of given design solutions and the impact of different parallel options on resource requirements, as demonstrated in MPSeeker [145,158], and [150]. Additionally, studies like Prospector[146] and Sherlock [148] achieve near-Pareto optimal configurations of the best design solutions at minimal costs through machine learning methods in design exploration strategy.

Despite their clear advantages in DSE, machine learning methods also face limitations. Firstly, they typically require extensive training data, necessitating retraining or redesign for emerging problems, increasing adaptation time and costs in new scenarios. Secondly, the interpretability of machine learning methods is poor, complicating the intuitive demonstration of the design exploration process and various design trade-offs, which is especially crucial in applications requiring a deep understanding of the reasons behind design decisions.

To overcome these limitations, researchers are developing new machine learning models and algorithms aimed at reducing reliance on extensive training data, enhancing model interpretability while maintaining efficiency in design optimization. For instance, by introducing

prior knowledge, employing semi-supervised learning, or reinforcement learning strategies, the models' generalization ability and adaptability can be improved with limited training data.

Reinforcement learning or heuristic algorithms, as trial-and-error optimization strategies, seek near-optimal solutions through iterative processes, showcasing powerful capabilities for handling large-scale design space problems. These methods stand out for their extreme flexibility and effectiveness, adapting to various complex design needs. Studies like IronMan [153,159,160]leverage the flexibility of reinforcement learning and heuristic algorithms to propose automated design frameworks based on different models, simplifying the design process and enhancing design efficiency. Moreover, research works such as [150–152] significantly reduce the time consumption of design exploration by effectively reusing existing design solutions or inferring new ones based on existing schemes. Furthermore, ERDSE [161] and ACDSE [156]introduce strategies like phase separation and pruning techniques to effectively balance the trade-off between local and global searches, reducing the risk of settling for local optima.

Despite the significant advantages of reinforcement learning and heuristic algorithms in DSE, they also present inherent randomness challenges, which may lead to search processes getting trapped in local optima and struggling to find global optimal solutions. Thus, selecting the appropriate optimization strategy for different design problems and scenarios becomes crucial. For problems with relatively smaller design spaces, traditional linear analysis or deterministic algorithms might be more applicable, offering stable and accurate solutions. Conversely, for problems with larger design spaces or those requiring handling highly complex, dynamically changing design conditions, machine learning methods, and reinforcement learning or heuristic algorithms exhibit higher applicability and flexibility.

4.2.3. Performance evaluation

In FPGA-based acceleration research for Neural Network Model, performance evaluation plays a crucial role. The primary goal of performance evaluation is to verify that hardware implementations meet the predefined performance metrics and resource limits, and to identify and mitigate any performance bottlenecks.This evaluation involves comprehensive benchmark testing of different design solutions, thoroughly assessing their computational speed, energy efficiency, and resource utilization efficiency, thereby providing a quantitative basis for the final hardware selection. The process aims to ensure optimization of the design, balancing performance with cost-effectiveness, and offers reliable evaluation criteria for the co-design of algorithms and hardware.

This study delineates the performance evaluation process into three core steps: establishing evaluation criteria, developing assessment models, and applying improvement strategies. Through these steps, researchers can not only ascertain the performance of each design solution but also identify potential optimization areas and directions for improvement. Hence, the research focus is on developing performance evaluation models suitable for different application scenarios and continually refining assessment methods to more accurately predict performance and propose effective performance improvement measures. For a deeper understanding of the methodologies and related research outcomes in performance evaluation, literature [163] to [164] provides detailed discussions and analyses. These references cover the theoretical foundations and practical examples of performance evaluation and showcase the latest research progress and technological innovations.

Performance evaluation plays a crucial role at various stages of FPGA hardware design, especially in design solutions aimed at accelerating Neural Network Model. This process includes simulating or emulating hardware circuits to verify the correctness of the design and its achieved performance metrics. By synthesizing key indicators such as delay, power consumption, and resource utilization, designers can effectively compare solutions and optimize performance.

The core objective of performance evaluation is to ensure that hardware circuits meet set performance metrics and resource constraints, while identifying and optimizing any potential performance bottlenecks. In FPGA-based hardware design projects, this process is critical for shortening design cycles, reducing design costs, and enhancing design quality and reliability.

However, FPGA platforms do not inherently offer built-in performance analysis tools, requiring designers to manually embed custom hardware monitors. Such monitors are typically inserted into communication channels or finite state machines to collect performance data. Although previous research has made progress in automating this process, these methods largely rely on low-level Hardware Description Languages (HDL), limiting deep insights into the root causes of performance issues (see Table 20).

In the process of Design Space Exploration (DSE) for accelerating Neural Network Model on FPGA platforms, performance evaluation of design solutions is crucial. The essence of performance evaluation lies in the comprehensive analysis of the efficiency and feasibility of design solutions through two main dimensions: resource consumption metrics and power consumption metrics. Resource consumption metrics focus on evaluating the efficiency of design solutions in terms of computational resources, storage resources, and communication resources utilization, while power consumption metrics concentrate on the energy performance of design solutions. Although these metrics focus on different aspects of performance, both are vital in ensuring the efficiency and practicality of design solutions.

1. Resource consumption and power consumption

In the exploration of Design Space Exploration (DSE) for accelerating Neural Network Model on FPGA platforms, the role of performance metrics is pivotal, providing a quantitative basis for the final hardware selection through comprehensive benchmarking tests of different design solutions. This process aims to ensure the optimization of the design, balancing performance with cost-effectiveness, and provides a reliable evaluation standard for the co-design of algorithms and hardware.

The performance evaluation process is divided into three core steps: determination of evaluation criteria, establishment of assessment models, and application of improvement strategies. Through these steps, researchers can not only determine the performance of each design solution but also identify potential areas for optimization and direction for improvements. Therefore, the focus of research is on developing performance evaluation models suitable for various application scenarios and continually optimizing evaluation methods to more accurately predict performance and propose effective performance improvement measures.

For further understanding of performance evaluation methodologies and related research findings, Refs. [163] to [164] provide detailed discussions and analyses. These references cover the theoretical foundations and application examples of performance evaluation, showcasing the latest research progress and technological innovations.

Performance evaluation plays a crucial role in all stages of FPGA hardware design, especially in design solutions aimed at accelerating deep learning models. This process encompasses simulating or emulating hardware circuits to verify the design's correctness and its achieved performance metrics. By analyzing key indicators such as latency, power consumption, and resource occupancy, designers can effectively compare schemes and optimize performance.

The core goal of performance evaluation is to ensure that the hardware circuit meets established performance metrics and resource constraints, while identifying and optimizing any potential performance bottlenecks. In FPGA hardware design projects, this process is crucial for shortening design cycles, reducing design costs, and improving design quality and reliability.

However, FPGA platforms do not offer built-in performance analysis tools, requiring designers to manually insert custom hardware monitors. Such monitors are typically inserted into communication channels

Table 20
Comparison of Performance Evaluation Methods.

Method name	Selection of indicators	Model building	Optimization methods	Realization details	Dominance	Inferior
[165]	Depletion of resources	Roof model	Linear analysis	PE replication factor; Expandable parameters	Integration of parameters and roof modeling	Dependence on quality of advanced synthesis reports
[166]	Power wastage	Experience roofing toolkit	Linear analysis	Energy Efficiency Benchmarking	Explore the benefits of dedicated computing	Discussion of scientific computing application scenarios only
Pyramid [167]	Depletion of resources	Machine learning model	Machine learning	Minerva Tools	Rapid estimation of resource consumption and timing	Requires a lot of data to train the model
HLScope [163]	Depletion of resources	Vivado-based Source-to-Source conversion framework	Linear analysis	Stop Analysis HLS Integration	Easy identification of performance bottlenecks and extraction parameters	Dependent on relevant technologies for advanced synthesis
FlexCL [168]	Power wastage	Power consumption modeling based on OpenCL loads	Linear analysis	Integrating Computation; A global memory model	Quickly Evaluate OpenCL Design Performance	OpenCL programming flow only
[169]	Depletion of resources	FPGA Experience Rooftop	Linear analysis	Customizable interconnect	Rapid assessment of performance limits	Discuss HPC application scenarios only
[170]	Composite indicators	Cost modeling for partial reconfiguration	Linear analysis	Partial reconstruction	Effective prediction of partially reconfigured system throughput	No discussion of the latency of each refactoring component
PCRAM [171]	Composite indicators	Theoretical computational modeling based on word-RAM	Sorting algorithm	Deriving lower bounds on running time	Consider heterogeneous system solutions	Need to optimize existing algorithms
PowerGear [172]	Power wastage	HEC-GNN	Linear analysis	Edge Semantics Structural Properties	Highly scalable power consumption estimation	Dependent on graph data construction
[173]	Composite indicators	OpenCL-based performance analysis model	Linear analysis	Four Interpretable Metrics	Can guide OpenCL code optimization	Dependent on static and dynamic analysis
[174]	Composite indicators	A restricted set of communication modes	Linear analysis	Synchronizing Data Streams Batch synchronization	Efficient evaluation of hardware resources and scheduling time	Limitations of current compiler support
Aladdin [175] HAPE [176]	Power wastage	Dynamic data dependency diagram	Linear analysis	Estimation at the code level	No RTL design required	Requires additional resources to make the call
KAPow [177] [178]	Power wastage	Model based on signaling activity	Linear analysis	Strong correlation modeling of activity and power	Avoid time-consuming characterization processes	Requires additional resources for online testing
[179,180]	Power wastage	Model based on signaling activity	Neural network approach	Strong correlation modeling of activity and power	Fast analog design	Errors in complex interconnections
[181]	Power wastage	Improved power estimation model	Nonlinear regression	Dunn's model; Clock Enable	Accurate estimation of power after clock enable optimization	Only one optimization technique, clock enable, was considered
[125]	Power wastage	Linear regression and multilayer perceptron	Generating approximate solutions	Model retraining; Model fitting	High power/area ratio	Additional exploration of model parameters is required
HLSPredect [164]	Power wastage	A framework for integration-based machine learning models	Multiple machine learning models	Instructionless and instructed IP; core hardware templates	Cross-platform assessment capabilities	Requires large amounts of training data
HL-Pow [182]	Power wastage	A Learning-Based Framework for Modeling Power Consumption	Machine learning	Feature Construction; Power harvesting	Overcoming the gap between synthesis and estimation	Dependent on the quality of the training data
[183]	Power wastage	Reverse propagation networks; A Statistical Approach	Heuristic Algorithms; A regression approach	Feature vector extraction; Communication-centered	Exploring heuristics and regression methods	Dependent characteristic vector quality

(*continued on next page*)

Table 20 (*continued*).

Method name	Selection of indicators	Model building	Optimization methods	Realization details	Dominance	Inferior
Minerva [184]	Composite indicators	C to FPGA results database	Deep Q Learning	Benchmarking; Adaptive constraints	High throughput/area ratio	Dependent on CAD toolset
IMap [185]	Hardware area	Side delay model	heuristic algorithm	Hardware area is prioritized; Depth prioritized	Iteratively collect data Guide the mapping process	Focus only on hardware area related optimizations

or finite state machines to collect performance data. Although previous research has made progress in automating this process, most methods rely on low-level Hardware Description Languages (HDL), limiting the ability to deeply understand the root causes of performance issues.

2. Customized analysis model

Beyond the Roofline model, researchers have proposed a variety of customized analysis models tailored to specific design needs and environments, considering different hardware characteristics, execution environments, hardware metrics, and design tools. For instance, Ref. [170]presents a cost model tailored for the Partial Reconfiguration (PR) feature on FPGAs through empirical research. The Pipelining Circuit RAM (PCRAM) model, introduced in Ref. [171], specifically analyzes theoretical models of system computations limited to synchronous circuit environments. Additionally, PowerGear [172]] and FlexCL [168] each designed different analysis models, offering optimization directions for specific design goals. Notably, in Ref. [168], FlexCL introduced a power consumption model for OpenCL workloads on FPGAs, modeling different global memory access patterns and effectively utilizing memory and computation in communication patterns for comprehensive assessment.

3. Linear analysis performance evaluation

In addition to Design Space Exploration (DSE), performance evaluation methods based on linear analysis play a crucial role. These methods typically encompass comprehensive evaluations at the code level, computation kernel level, and computation level, aiming to provide researchers with accurate information on performance, power consumption, and resource utilization. Through such methods, researchers can predict and optimize the performance of Neural Network Model on FPGAs during the early design stages.

At the code level, some tools and approaches focus on performing performance evaluations directly from the source code level or converting the code into other representations for easier evaluation. For example, studies [173], Aladdin [175], HAPE [176], and [180] demonstrate how to directly assess performance by analyzing source code or converting it into intermediate representations for deeper analysis. The advantage of this method is its ability to identify potential performance bottlenecks and optimization points early on, but it may require a high understanding of the code and conversion costs.

At the computation kernel level, tools such as HLScope [163,178], and FlexCL [168] focus on taking computation kernels as input to enhance the accuracy of performance evaluation. These methods provide performance predictions for specific algorithms' hardware implementations by analyzing specific computation modules. FlexCL [154] specifically proposed a power consumption model for FPGA-based OpenCL workloads, conducting a comprehensive assessment through modeling global memory access patterns and effective utilization of communication patterns and computation.

At the computation level, other methods like KAPow [177], [179], and [174] provide performance and resource consumption estimates based on fixed communication or computation rules. This approach simplifies the evaluation process through predefined rules, enabling researchers to quickly obtain estimates of key performance indicators, thereby guiding subsequent design and optimization efforts. 4.Evaluation methods based on machine learning and reinforcement learning

In the research of deep learning model acceleration techniques on FPGA platforms, exploring the complex relationships between different hardware design options and their outcomes is key to achieving optimized designs. In recent years, methods based on machine learning and reinforcement learning have been widely applied in this field, focusing on intuitively constructing the association between design choice selections and performance outcomes through training data and modeling processes.

Machine Learning-based Methods: Pyramid [167] demonstrates the capability of machine learning methods in establishing relationships between design choices and outcomes by directly training the mapping from C code to FPGA. This approach can effectively identify optimal design solutions, reducing development cycles. HLSPredict [164] analyzes the relationship between different computing kernels and performance or power consumption indicators, predicting the performance and power consumption of different design options through a machine learning model, providing guidance for design optimization. Study [125] utilizes machine learning-constructed relationships to generate approximate solutions aiming to find lower power consumption optimization strategies, showcasing the practicality of machine learning methods in assisting design space exploration.

Reinforcement Learning and Heuristic Methods: Research [183] compares the effectiveness of heuristic methods and statistical methods in predicting FPGA power consumption, confirming the efficacy of prior knowledge and heuristic rules in improving evaluation quality. Minerva [184] proposes constructing the mapping from C code to FPGA outcomes based on constraints in four aspects, including resource usage, latency, power consumption, and area, highlighting the importance of prior knowledge in the design process. Another study [185] offers different patterns to establish the mapping between computing kernels and power consumption, revealing the application potential of heuristic algorithms in design optimization.

These advancements demonstrate the integration of advanced computational models and traditional design techniques, emphasizing the importance of machine learning, reinforcement learning, and heuristic approaches in navigating the vast design space of FPGA-based deep learning accelerations. By leveraging these methods, researchers and designers can achieve more efficient and effective optimization strategies, tailoring their approaches to the unique demands of each application and hardware configuration.

5. Future trends and challenges

5.1. Emerging technologies and methods

As artificial intelligence technology continues to advance, FPGA have become increasingly crucial in the AI domain due to their exceptional performance and flexibility. FPGAs offer tailor-made hardware acceleration solutions, especially critical for deep learning tasks requiring rapid processing and efficiency. A variety of emerging technologies and methodologies are anticipated to further propel the application and development of FPGAs within AI.

Initially, the integration of quantum computing with FPGAs heralds the emergence of a new computing paradigm. Quantum computing, known for its extraordinary computational capabilities for certain specific problems, coupled with the flexibility and programmability of FPGAs, presents an ideal platform for exploring quantum algorithms and implementing quantum computing simulations. By simulating quantum computing processes on FPGAs, researchers can develop and test

quantum algorithms without relying on actual quantum hardware, accelerating the research and application of quantum computing. Additionally, FPGAs can serve as a bridge between quantum computing and classical computing, handling tasks inefficiently processed by quantum computers, or working collaboratively to enhance the overall system's computational capacity and efficiency.

With the proliferation of the Internet of Things (IoT) and intelligent devices, near-edge computing has emerged as a significant trend. Edge computing, by processing data close to its origin, aims to reduce latency, enhance response speeds, and bolster data security and privacy protection. FPGAs hold unique advantages in this domain; their high performance and low latency make them highly suitable for deployment on edge devices to execute complex Neural Network Model and AI algorithms. Moreover, the reconfigurability of FPGAs allows for algorithm updates or optimizations without hardware replacement, significantly enhancing the flexibility and long-term effectiveness of edge devices.

Another direction of interest is the development of adaptive AI models. As the complexity and diversity of AI applications continue to grow, intelligent models capable of autonomously adapting to varying data flows and computational demands are needed. FPGAs, with their high configurability and adaptability, provide an ideal hardware foundation for developing and deploying such adaptive AI models. By real-time monitoring of data flow and computational load, FPGAs can dynamically adjust hardware resource allocation and computational strategies, enabling more efficient and intelligent data processing. This hardware-level adaptability opens new possibilities for the real-time optimization and personalized application of AI models.

5.2. Potential applications of FPGA acceleration in the AI domain

With the advancement of Artificial Intelligence (AI), FPGA are becoming increasingly critical in the AI domain, particularly in high-demand scenarios such as autonomous driving systems, intelligent medical diagnostics, and smart video surveillance.

One of the primary challenges in autonomous driving technology is the real-time processing of vast amounts of data from various sensors and making swift, accurate decisions based on this data. FPGAs demonstrate considerable potential in this field, capable of parallel processing signals from multiple sensors, including cameras, radars, and Light Detection and Ranging (LiDAR), to achieve real-time perception of the surrounding environment. Additionally, FPGAs offer efficient support for executing deep learning algorithms, such as Convolutional Neural Networks (CNNs) for visual recognition and object detection. Their low latency and high throughput make FPGAs an indispensable component in vehicular systems, providing real-time data processing and decision-making support to enhance driving safety and comfort in autonomous vehicles.

In the realm of intelligent medical diagnostics, the application of FPGAs is increasingly widespread, especially in accelerating medical image processing and analysis. Utilizing FPGAs to process medical images, such as MRI and CT scans, significantly enhances the speed of image resolution, allowing doctors to obtain diagnostic results more swiftly. This is particularly vital in time-sensitive medical situations, such as emergency and intensive care. Moreover, the flexibility of FPGAs enables adjustment of algorithms according to various diseases and diagnostic needs, supporting the formulation of personalized medical plans. By optimizing Neural Network Model at the hardware level, such as neural networks for image classification and segmentation, FPGAs can help improve the accuracy and efficiency of disease diagnostics, thereby enhancing patient treatment outcomes and quality of life.

Smart video surveillance systems serve as essential tools for public safety. The high computational capability of FPGAs enables real-time processing of video streams from multiple cameras, conducting complex video analysis tasks such as facial recognition and behavior analysis. Compared to traditional video surveillance systems, smart video surveillance systems accelerated by FPGAs can immediately identify suspicious behaviors or unusual events, quickly triggering alerts and significantly improving the efficiency and responsiveness of public safety systems. For instance, in densely populated public areas, FPGAs can assist in analyzing crowd density and detecting abnormal behaviors, providing robust technical support for security management. Furthermore, the low-power characteristic of FPGAs also enables the deployment of smart video surveillance systems in edge computing environments, making data processing more localized, further reducing latency, and enhancing the system's real-time and reliability.

5.3. Challenges and directions for development

Recent advancements in neural network acceleration and optimization on FPGA platforms have marked significant milestones in the field. Despite these achievements, challenges persist due to the continuous evolution of Neural Network Model and diverse application contexts. Future directions for FPGA-based neural network acceleration and optimization techniques can be summarized as follows:

1. Integrated Software-Hardware Design and Description: The primary challenge in co-design lies in establishing an efficient and flexible framework for seamless integration of software and hardware design, maintaining high performance and low power consumption. This necessitates the exploration of new design languages and tools capable of describing both software logic and hardware architecture. Additionally, the development of intelligent compilers and optimization tools for automatic optimal resource allocation and scheduling between software and hardware is imperative.

2. Innovative Memory Architectures and Data Management Strategies: Designing memory architectures and data flow management strategies that meet high bandwidth and low latency requirements, with scalability and flexibility, is crucial. Efficient caching strategies and data prefetching techniques are essential to reduce memory access latency. Exploring heterogeneous storage solutions that combine DRAM, SRAM, and non-volatile memory technologies could optimize data access speed and storage capacity.

3. Hardware-Oriented Neural Network Architectures: Existing neural network models may not fully exploit FPGA characteristics. Identifying or designing efficient network structures suited for specific hardware platforms (e.g., FPGAs) while maintaining model accuracy is vital. Utilizing hardware-aware Neural Architecture Search (NAS) methods to automatically find models adapted to specific hardware features, combined with model compression and quantization techniques, can further enhance model execution efficiency and reduce resource consumption.

4. Development of Integrated Tools and Performance Evaluation Models: There is a lack of effective tools and models for evaluating AI model performance and resource consumption on FPGAs. However, developing integrated tools and models supporting the entire process from design to deployment, providing accurate performance evaluations and optimization recommendations, is a complex and challenging task. By developing advanced synthesis tools for co-design and automatic optimization, and constructing comprehensive performance evaluation models to accurately predict the impact of different design choices on performance, informed design decisions can be guided.

5. Synthesizing Generalized and Customized Designs: Finding the optimal balance between the flexibility of generalized designs and the efficiency of customized designs to accommodate the varying demands of application requirements and hardware platform limitations is essential. Exploring new design paradigms and methods for modularity and configurability of designs enables rapid adaptation to different models and tasks. Furthermore, developing intelligent optimization algorithms that automatically select or adjust design strategies based on the application scenario can achieve optimal performance and resource utilization.

6. Summary

In the field of deep learning model application and deployment, FPGA-based neural network design plays a crucial role. Despite FPGA's excellent resource utilization efficiency and high flexibility, the difference between model design and hardware design makes FPGA-accelerated neural network systems a challenging task. Although existing literature extensively introduces related technologies and design methods, they often lack a holistic analysis of these solutions.

This survey delves into the acceleration technology of Neural Network Model based on FPGA, covering from the introduction of research background and significance to the principles of FPGA technology, comparison with other computing platforms, and discussions oriented towards specific applications and general needs, establishing a solid theoretical foundation. The article thoroughly analyzes technological implementation methods, including algorithm-level optimizations such as sparsification, model structure adjustments, low-rank approximations, and quantization strategies, as well as hardware-level optimizations including data flow optimization and parallel computing architectures. Additionally, the importance of compiler optimization is discussed, illustrating the role of Register Transfer Level (RTL), High-Level Synthesis (HLS), and mixed methods in improving design efficiency. Particularly, in the section on co-optimization methods of algorithms and hardware, the paper further expounds on the challenges, implementation strategies, and key steps of co-design, highlighting the importance of algorithm mapping, design space exploration, and performance evaluation, revealing the optimization framework and specific implementation paths of integrating algorithms and hardware.

This paper provides a comprehensive overview of technologies and optimization strategies for FPGA in the AI acceleration application field. By discussing the co-design of algorithm optimization and hardware optimization, it points out the key pathways to enhancing acceleration capabilities. Despite numerous challenges, with technological innovation and interdisciplinary collaboration, the deep learning acceleration capability of FPGAs is expected to continuously strengthen, promoting the development of efficient and intelligent computing applications.

CRediT authorship contribution statement

Fang Liu: Writing – review & editing, Writing – original draft, Methodology, Investigation, Data curation, Conceptualization. **Heyuan Li:** Writing – review & editing. **Wei Hu:** Writing – review & editing, Methodology, Investigation, Data curation, Conceptualization. **Yanxiang He:** Writing – review & editing, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This paper is supported by the National Key Research and Development Program of China (No. 2021ZD0113304), National Natural Science Foundation of China (U23A20316), Key R&D Project of Hubei Province (2021BAA029), the General Program of Natural Science Foundation of China (NSFC) (No. 62072346, No. 61972293), Hubei Higher Education Excellent Youth Science and Technology Innovation Team Project (No. T2022060) and Wuhan City College Research Science Project (2023CYYYJJ01).

References

[1] S. Gao, X. Liu, B. Zeng, S. Xu, Y. Li, X. Luo, J. Liu, X. Zhen, B. Zhang, Implicit diffusion models for continuous super-resolution, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 10021–10030.
[2] B. Fei, Z. Lyu, L. Pan, J. Zhang, W. Yang, T. Luo, B. Zhang, B. Dai, Generative diffusion prior for unified image restoration and enhancement, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 9935–9946.
[3] X.-B. Nguyen, C.N. Duong, X. Li, S. Gauch, H.-S. Seo, K. Luu, Micron-bert: Bert-based facial micro-expression recognition, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 1482–1492.
[4] S. Han, H. Mao, W.J. Dally, Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding, 2015, arXiv preprint arXiv:1510.00149.
[5] Y. Guo, A. Yao, Y. Chen, Dynamic network surgery for efficient dnns, in: Advances in Neural Information Processing Systems, vol. 29, 2016.
[6] Z. Liu, J. Xu, X. Peng, R. Xiong, Frequency-domain dynamic pruning for convolutional neural networks, in: Advances in Neural Information Processing Systems, vol. 31, 2018.
[7] J. Chang, J. Sha, Prune deep neural networks with the modified $l_{1/2}$ penalty, IEEE Access 7 (2018) 2273–2280.
[8] Z. Wang, K. Xu, S. Wu, L. Liu, L. Liu, D. Wang, Sparse-YOLO: Hardware/software co-design of an FPGA accelerator for YOLOv2, IEEE Access 8 (2020) 116569–116585.
[9] V. Lebedev, V. Lempitsky, Fast convnets using group-wise brain damage, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 2554–2564.
[10] J. Faraone, G. Gambardella, D. Boland, N. Fraser, M. Blott, P.H. Leong, Customizing low-precision deep neural networks for FPGAs, in: 2018 28th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2018, pp. 97–973.
[11] H. Li, A. Kadav, I. Durdanovic, H. Samet, H.P. Graf, Pruning filters for efficient convnets, 2016, arXiv preprint arXiv:1608.08710.
[12] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, C. Zhang, Learning efficient convolutional networks through network slimming, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 2736–2744.
[13] A. Polyak, L. Wolf, Channel-level acceleration of deep face representations, IEEE Access 3 (2015) 2163–2175.
[14] H.-J. Kang, Accelerator-aware pruning for convolutional neural networks, IEEE Trans. Circuits Syst. Video Technol. 30 (7) (2019) 2093–2103.
[15] C. Buciluǎ, R. Caruana, A. Niculescu-Mizil, Model compression, in: Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2006, pp. 535–541.
[16] J. Ba, R. Caruana, Do deep nets really need to be deep? in: Advances in Neural Information Processing Systems, vol. 27, 2014.
[17] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, 2015, arXiv preprint arXiv:1503.02531.
[18] T. Chen, I. Goodfellow, J. Shlens, Net2net: Accelerating learning via knowledge transfer, 2015, arXiv preprint arXiv:1511.05641.
[19] J. Shen, N. Vesdapunt, V.N. Boddeti, K.M. Kitani, In teacher we trust: Learning compressed models for pedestrian detection, 2016, arXiv preprint arXiv:1612.00478.
[20] G. Chen, W. Choi, X. Yu, T. Han, M. Chandraker, Learning efficient object detection models with knowledge distillation, in: Advances in Neural Information Processing Systems, vol. 30, 2017.
[21] V. Sanh, L. Debut, J. Chaumond, T. Wolf, DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, 2019, arXiv preprint arXiv:1910.01108.
[22] S. Zagoruyko, N. Komodakis, Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer, 2016, arXiv preprint arXiv:1612.03928.
[23] C. Yang, L. Xie, S. Qiao, A. Yuille, Knowledge distillation in generations: More tolerant teachers educate better students, 2018, arXiv preprint arXiv:1805.05551.
[24] Y. Zhang, T. Xiang, T.M. Hospedales, H. Lu, Deep mutual learning, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 4320–4328.
[25] X. Ding, Y. Guo, G. Ding, J. Han, Acnet: Strengthening the kernel skeletons for powerful CNN via asymmetric convolution blocks, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 1911–1920.
[26] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, J. Sun, Repvgg: Making VGG-style convnets great again, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 13733–13742.
[27] M. Zhang, X. Yu, J. Rong, L. Ou, Repnas: Searching for efficient re-parameterizing blocks, 2021, arXiv preprint arXiv:2109.03508.
[28] X. Ding, T. Hao, J. Tan, J. Liu, J. Han, Y. Guo, G. Ding, Resrep: Lossless CNN pruning via decoupling remembering and forgetting, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 4510–4520.

[29] X. Ding, C. Xia, X. Zhang, X. Chu, J. Han, G. Ding, Repmlp: Re-parameterizing convolutions into fully-connected layers for image recognition, 2021, arXiv preprint arXiv:2105.01883.

[30] X. Ding, X. Zhang, J. Han, G. Ding, Diverse branch block: Building a convolution as an inception-like unit, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 10886–10895.

[31] F.N. Iandola, S. Han, M.W. Moskewicz, K. Ashraf, W.J. Dally, K. Keutzer, SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and< 0.5 MB model size, 2016, arXiv preprint arXiv:1602.07360.

[32] A.G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, H. Adam, Mobilenets: Efficient convolutional neural networks for mobile vision applications, 2017, arXiv preprint arXiv:1704.04861.

[33] X. Zhang, X. Zhou, M. Lin, J. Sun, Shufflenet: An extremely efficient convolutional neural network for mobile devices, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 6848–6856.

[34] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 1–9.

[35] S. Ioffe, C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: International Conference on Machine Learning, pmlr, 2015, pp. 448–456.

[36] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, Rethinking the inception architecture for computer vision, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 2818–2826.

[37] C. Szegedy, S. Ioffe, V. Vanhoucke, A. Alemi, Inception-v4, inception-resnet and the impact of residual connections on learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 31, (no. 1) 2017.

[38] S. Mehta, M. Rastegari, A. Caspi, L. Shapiro, H. Hajishirzi, Espnet: Efficient spatial pyramid of dilated convolutions for semantic segmentation, in: Proceedings of the European Conference on Computer Vision, ECCV, 2018, pp. 552–568.

[39] S. Mehta, M. Rastegari, L. Shapiro, H. Hajishirzi, Espnetv2: A light-weight, power efficient, and general purpose convolutional neural network, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 9190–9200.

[40] I. Oguntola, S. Olubeko, C. Sweeney, Slimnets: An exploration of deep model compression and acceleration, in: 2018 IEEE High Performance Extreme Computing Conference, HPEC, IEEE, 2018, pp. 1–6.

[41] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, Mobilenetv2: Inverted residuals and linear bottlenecks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 4510–4520.

[42] B. Koonce, B. Koonce, MobileNetV3, in: Convolutional Neural Networks with Swift for Tensorflow: Image Recognition and Dataset Categorization, Springer, 2021, pp. 125–144.

[43] N. Ma, X. Zhang, H.-T. Zheng, J. Sun, Shufflenet v2: Practical guidelines for efficient CNN architecture design, in: Proceedings of the European Conference on Computer Vision, ECCV, 2018, pp. 116–131.

[44] A. Tommasel, D. Godoy, A social-aware online short-text feature selection technique for social media, Inf. Fusion 40 (2018) 1–17.

[45] E.L. Denton, W. Zaremba, J. Bruna, Y. LeCun, R. Fergus, Exploiting linear structure within convolutional networks for efficient evaluation, in: Advances in Neural Information Processing Systems, vol. 27, 2014.

[46] E. Wang, J.J. Davis, R. Zhao, H.-C. Ng, X. Niu, W. Luk, P.Y. Cheung, G.A. Constantinides, Deep neural network approximation for custom hardware: Where we've been, where we're going, ACM Comput. Surv. 52 (2) (2019) 1–39.

[47] G. Hegde, Siddhartha, N. Kapre, CaffePresso: Accelerating convolutional networks on embedded SoCs, ACM Trans. Embedd. Comput. Syst. (TECS) 17 (1) (2017) 1–26.

[48] D.T. Nguyen, T.N. Nguyen, H. Kim, H.-J. Lee, A high-throughput and power-efficient FPGA implementation of YOLO CNN for object detection, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 27 (8) (2019) 1861–1873.

[49] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, et al., Dadiannao: A machine-learning supercomputer, in: 2014 47th Annual IEEE/ACM International Symposium on Microarchitecture, IEEE, 2014, pp. 609–622.

[50] T. Dettmers, 8-bit approximations for parallelism in deep learning, 2015, arXiv preprint arXiv:1511.04561.

[51] G. Chen, H. Meng, Y. Liang, K. Huang, GPU-accelerated real-time stereo estimation with binary neural network, IEEE Trans. Parallel Distrib. Syst. 31 (12) (2020) 2896–2907.

[52] S. Gupta, A. Agrawal, K. Gopalakrishnan, P. Narayanan, Deep learning with limited numerical precision, in: International Conference on Machine Learning, PMLR, 2015, pp. 1737–1746.

[53] C. Zhang, D. Wu, J. Sun, G. Sun, G. Luo, J. Cong, Energy-efficient CNN implementation on a deeply pipelined fpga cluster, in: Proceedings of the 2016 International Symposium on Low Power Electronics and Design, 2016, pp. 326–331.

[54] S. Liang, S. Yin, L. Liu, W. Luk, S. Wei, FP-BNN: Binarized neural network on FPGA, Neurocomputing 275 (2018) 1072–1086.

[55] Z. He, D. Fan, Simultaneously optimizing weight and quantizer of ternary neural network using truncated gaussian approximation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 11438–11446.

[56] C. Zhu, S. Han, H. Mao, W.J. Dally, Trained ternary quantization, 2016, arXiv preprint arXiv:1612.01064.

[57] E.H. Lee, D. Miyashita, E. Chai, B. Murmann, S.S. Wong, Lognet: Energy-efficient neural networks using logarithmic computation, in: 2017 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, IEEE, 2017, pp. 5900–5904.

[58] V. Sze, Y.-H. Chen, T.-J. Yang, J.S. Emer, Efficient processing of deep neural networks: A tutorial and survey, Proc. IEEE 105 (12) (2017) 2295–2329.

[59] J. Yang, X. Shen, J. Xing, X. Tian, H. Li, B. Deng, J. Huang, X.-s. Hua, Quantization networks, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 7308–7316.

[60] D. Miyashita, E.H. Lee, B. Murmann, Convolutional neural networks using logarithmic data representation, 2016, arXiv preprint arXiv:1603.01025.

[61] M. Rastegari, V. Ordonez, J. Redmon, A. Farhadi, Xnor-net: Imagenet classification using binary convolutional neural networks, in: European Conference on Computer Vision, Springer, 2016, pp. 525–542.

[62] S.I. Venieris, A. Kouris, C.-S. Bouganis, Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions, ACM Comput. Surv. 51 (3) (2018) 1–39.

[63] B. Li, S. Pandey, H. Fang, Y. Lyv, J. Li, J. Chen, M. Xie, L. Wan, H. Liu, C. Ding, Ftrans: energy-efficient acceleration of transformers using FPGA, in: Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design, 2020, pp. 175–180.

[64] L. Yin, R. Cheng, Y. Wen, C. Liu, J. He, Emerging 2D memory devices for in-memory computing, Adv. Mater. 33 (29) (2021) 2007081.

[65] J. Mair, Z. Huang, D. Eyers, Y. Chen, Quantifying the energy efficiency challenges of achieving exascale computing, in: 2015 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, IEEE, 2015, pp. 943–950.

[66] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, J. Cong, Optimizing FPGA-based accelerator design for deep convolutional neural networks, in: Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2015, pp. 161–170.

[67] H. Dong, L. Jiang, T. Li, X. Liang, A systematic FPGA acceleration design for applications based on convolutional neural networks, in: AIP Conference Proceedings, vol. 1955, (no. 1) AIP Publishing, 2018.

[68] C. Zhang, G. Sun, Z. Fang, P. Zhou, P. Pan, J. Cong, Caffeine: Toward uniformed representation and acceleration for deep convolutional neural networks, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 38 (11) (2018) 2072–2085.

[69] Y. Ma, Y. Cao, S. Vrudhula, J.-s. Seo, Optimizing the convolution operation to accelerate deep neural networks on FPGA, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 26 (7) (2018) 1354–1367.

[70] A. Rahman, J. Lee, K. Choi, Efficient FPGA acceleration of convolutional neural networks using logical-3D compute array, in: 2016 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE, 2016, pp. 1393–1398.

[71] B. Akin, F. Franchetti, J.C. Hoe, Data reorganization in memory using 3D-stacked DRAM, ACM SIGARCH Comput. Archit. News 43 (3S) (2015) 131–143.

[72] M. Irfan, Z. Ullah, R.C. Cheung, D-TCAM: A high-performance distributed RAM based TCAM architecture on FPGAs, IEEE Access 7 (2019) 96060–96069.

[73] M. Alwani, H. Chen, M. Ferdman, P. Milder, Fused-layer CNN accelerators, in: 2016 49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO, IEEE, 2016, pp. 1–12.

[74] R. Zhao, H.-C. Ng, W. Luk, X. Niu, Towards efficient convolutional neural network for domain-specific applications on FPGA, in: 2018 28th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2018, pp. 147–1477.

[75] K. Seto, H. Nejatollahi, J. An, S. Kang, N. Dutt, Small memory footprint neural network accelerators, in: 20th International Symposium on Quality Electronic Design, ISQED, IEEE, 2019, pp. 253–258.

[76] H. Li, X. Fan, L. Jiao, W. Cao, X. Zhou, L. Wang, A high performance FPGA-based accelerator for large-scale convolutional neural networks, in: 2016 26th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2016, pp. 1–9.

[77] X. Wei, C.H. Yu, P. Zhang, Y. Chen, Y. Wang, H. Hu, Y. Liang, J. Cong, Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs, in: Proceedings of the 54th Annual Design Automation Conference 2017, 2017, pp. 1–6.

[78] A. Farmahini-Farahani, J.H. Ahn, K. Morrow, N.S. Kim, DRAMA: An architecture for accelerated processing near memory, IEEE Comput. Archit. Lett. 14 (1) (2014) 26–29.

[79] S. Ghaffari, S. Sharifian, FPGA-based convolutional neural network accelerator design using high level synthesize, in: 2016 2nd International Conference of Signal Processing and Intelligent Systems, ICSPIS, IEEE, 2016, pp. 1–6.

[80] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, K. Keutzer, Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 10734–10742.

[81] H. Cai, L. Zhu, S. Han, Proxylessnas: Direct neural architecture search on target task and hardware, 2018, arXiv preprint arXiv:1812.00332.

[82] Z. Guo, X. Zhang, H. Mu, W. Heng, Z. Liu, Y. Wei, J. Sun, Single path one-shot neural architecture search with uniform sampling, in: Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XVI 16, Springer, 2020, pp. 544–560.

[83] D. Stamoulis, R. Ding, D. Wang, D. Lymberopoulos, B. Priyantha, J. Liu, D. Marculescu, Single-path NAS: Designing hardware-efficient convnets in less than 4 hours, in: Joint European Conference on Machine Learning and Knowledge Discovery in Databases, Springer, 2019, pp. 481–497.

[84] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, Q.V. Le, Mnasnet: Platform-aware neural architecture search for mobile, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 2820–2828.

[85] W. Jiang, X. Zhang, E.H.-M. Sha, L. Yang, Q. Zhuge, Y. Shi, J. Hu, Accuracy vs. efficiency: Achieving both through fpga-implementation aware neural architecture search, in: Proceedings of the 56th Annual Design Automation Conference 2019, 2019, pp. 1–6.

[86] Y. Li, C. Hao, X. Zhang, X. Liu, Y. Chen, J. Xiong, W.-m. Hwu, D. Chen, Edd: Efficient differentiable dnn architecture and implementation co-search for embedded AI solutions, in: 2020 57th ACM/IEEE Design Automation Conference, DAC, IEEE, 2020, pp. 1–6.

[87] A. Marchisio, A. Massa, V. Mrazek, B. Bussolino, M. Martina, M. Shafique, NASCaps: A framework for neural architecture search to optimize the accuracy and hardware efficiency of convolutional capsule networks, in: Proceedings of the 39th International Conference on Computer-Aided Design, 2020, pp. 1–9.

[88] P. Achararit, M.A. Hanif, R.V.W. Putra, M. Shafique, Y. Hara-Azumi, APNAS: Accuracy-and-performance-aware neural architecture search for neural hardware accelerators, IEEE Access 8 (2020) 165319–165334.

[89] W. Jiang, L. Yang, S. Dasgupta, J. Hu, Y. Shi, Standing on the shoulders of giants: Hardware and neural architecture co-search with hot start, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 39 (11) (2020) 4154–4165.

[90] L.L. Zhang, Y. Yang, Y. Jiang, W. Zhu, Y. Liu, Fast hardware-aware neural architecture search, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 2020, pp. 692–693.

[91] W. Jiang, L. Yang, E.H.-M. Sha, Q. Zhuge, S. Gu, S. Dasgupta, Y. Shi, J. Hu, Hardware/software co-exploration of neural architectures, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 39 (12) (2020) 4805–4815.

[92] S. Zeng, H. Sun, Y. Xing, X. Ning, Y. Shan, X. Chen, Y. Wang, H. Yang, Black box search space profiling for accelerator-aware neural architecture search, in: 2020 25th Asia and South Pacific Design Automation Conference, ASP-DAC, IEEE, 2020, pp. 518–523.

[93] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, T. Darrell, Caffe: Convolutional architecture for fast feature embedding, in: Proceedings of the 22nd ACM International Conference on Multimedia, 2014, pp. 675–678.

[94] S. Guo, L. Wang, B. Chen, Q. Dou, Y. Tang, Z. Li, Fixcaffe: Training CNN with low precision arithmetic operations by fixed point caffe, in: Advanced Parallel Processing Technologies: 12th International Symposium, APPT 2017, Santiago de Compostela, Spain, August 29, 2017, Proceedings 12, Springer, 2017, pp. 38–50.

[95] P. Haglund, O. Mencer, W. Luk, B. Tai, Hardware design with a scripting language, in: Field Programmable Logic and Application: 13th International Conference, FPL 2003, Lisbon, Portugal, September 1-3, 2003 Proceedings 13, Springer, 2003, pp. 1040–1043.

[96] J. Decaluwe, MyHDL: A Python-based hardware description language, Linux J. 2004 (127) (2004) 5.

[97] D. Lockhart, G. Zibrat, C. Batten, PyMTL: A unified framework for vertically integrated computer architecture research, in: 2014 47th Annual IEEE/ACM International Symposium on Microarchitecture, IEEE, 2014, pp. 280–292.

[98] J. Clow, G. Tzimpragos, D. Dangwal, S. Guo, J. McMahan, T. Sherwood, A pythonic approach for rapid hardware prototyping and instrumentation, in: 2017 27th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2017, pp. 1–7.

[99] P. Bellows, B. Hutchings, JHDL-an HDL for reconfigurable systems, in: Proceedings. IEEE Symposium on FPGAs for Custom Computing Machines (Cat. No. 98TB100251), IEEE, 1998, pp. 175–184.

[100] Y. Mei, L. Tang, Q. Xiao, Z. Zhang, Z. Zhang, J. Zang, J. Zhou, Y. Wang, W. Wang, M. Ren, Reconstituted high density lipoprotein (rHDL), a versatile drug delivery nanoplatform for tumor targeted therapy, J. Mater. Chem. B 9 (3) (2021) 612–633.

[101] R. Cieszewski, K. Pozniak, R. Romaniuk, Python based high-level synthesis compiler, in: Photonics Applications in Astronomy, Communications, Industry, and High-Energy Physics Experiments 2014, vol. 9290, SPIE, 2014, pp. 981–988.

[102] R. Xiao, J. Shi, C. Zhang, FPGA implementation of CNN for handwritten digit recognition, in: 2020 IEEE 4th Information Technology, Networking, Electronic and Automation Control Conference, ITNEC, vol. 1, IEEE, 2020, pp. 1128–1133.

[103] A. Prost-Boucle, A. Bourge, F. Pétrot, H. Alemdar, N. Caldwell, V. Leroy, Scalable high-performance architecture for convolutional ternary neural networks on FPGA, in: 2017 27Th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2017, pp. 1–7.

[104] Y. Wang, J. Xu, Y. Han, H. Li, X. Li, DeepBurning: Automatic generation of FPGA-based learning accelerators for the neural network family, in: Proceedings of the 53rd Annual Design Automation Conference, 2016, pp. 1–6.

[105] P. Xu, X. Zhang, C. Hao, Y. Zhao, Y. Zhang, Y. Wang, C. Li, Z. Guan, D. Chen, Y. Lin, AutoDNNchip: An automated DNN chip predictor and builder for both FPGAs and ASICs, in: Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2020, pp. 40–50.

[106] J. Xu, Z. Liu, J. Jiang, Y. Dou, S. Li, CaFPGA: An automatic generation model for CNN accelerator, Microprocess. Microsyst. 60 (2018) 196–206.

[107] H. Sharma, J. Park, D. Mahajan, E. Amaro, J.K. Kim, C. Shao, A. Mishra, H. Esmaeilzadeh, From high-level deep neural models to FPGAs, in: 2016 49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO, IEEE, 2016, pp. 1–12.

[108] M. Motamedi, P. Gysel, S. Ghiasi, PLACID: A platform for FPGA-based accelerator creation for DCNNs, ACM Trans. Multimed. Comput. Commun. Appl. (TOMM) 13 (4) (2017) 1–21.

[109] X. Zhang, J. Wang, C. Zhu, Y. Lin, J. Xiong, W.-m. Hwu, D. Chen, DNNBuilder: An automated tool for building high-performance DNN hardware accelerators for FPGAs, in: 2018 IEEE/ACM International Conference on Computer-Aided Design, ICCAD, IEEE, 2018, pp. 1–8.

[110] T. Posewsky, D. Ziener, A flexible FPGA-based inference architecture for pruned deep neural networks, in: Architecture of Computing Systems–ARCS 2018: 31st International Conference, Braunschweig, Germany, April 9–12, 2018, Proceedings 31, Springer, 2018, pp. 311–323.

[111] J. Shen, Y. Huang, Z. Wang, Y. Qiao, M. Wen, C. Zhang, Towards a uniform template-based architecture for accelerating 2D and 3D CNNs on FPGA, in: Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2018, pp. 97–106.

[112] Z. Liu, P. Chow, J. Xu, J. Jiang, Y. Dou, J. Zhou, A uniform architecture design for accelerating 2D and 3D CNNs on FPGAs, Electronics 8 (1) (2019) 65.

[113] Y. Umuroglu, N.J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, K. Vissers, Finn: A framework for fast, scalable binarized neural network inference, in: Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2017, pp. 65–74.

[114] S.I. Venieris, C.-S. Bouganis, fpgaConvNet: A framework for mapping convolutional neural networks on FPGAs, in: 2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines, FCCM, IEEE, 2016, pp. 40–47.

[115] H. Li, Z. Lin, X. Shen, J. Brandt, G. Hua, A convolutional neural network cascade for face detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 5325–5334.

[116] M. Motamedi, P. Gysel, V. Akella, S. Ghiasi, Design space exploration of FPGA-based deep convolutional neural networks, in: 2016 21st Asia and South Pacific Design Automation Conference, ASP-DAC, IEEE, 2016, pp. 575–580.

[117] X. Zhang, X. Liu, A. Ramachandran, C. Zhuge, S. Tang, P. Ouyang, Z. Cheng, K. Rupnow, D. Chen, High-performance video content recognition with long-term recurrent convolutional network for FPGA, in: 2017 27th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2017, pp. 1–4.

[118] K. Guo, L. Sui, J. Qiu, J. Yu, J. Wang, S. Yao, S. Han, Y. Wang, H. Yang, Angel-eye: A complete design flow for mapping CNN onto embedded FPGA, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 37 (1) (2017) 35–47.

[119] Y. Guan, H. Liang, N. Xu, W. Wang, S. Shi, X. Chen, G. Sun, W. Zhang, J. Cong, FP-DNN: An automated framework for mapping deep neural networks onto FPGAs with RTL-HLS hybrid templates, in: 2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines, FCCM, IEEE, 2017, pp. 152–159.

[120] Y. Ma, N. Suda, Y. Cao, S. Vrudhula, J.-s. Seo, ALAMO: FPGA acceleration of deep learning algorithms with a modularized RTL compiler, Integration 62 (2018) 14–23.

[121] Y.-H. Lai, Y. Chi, Y. Hu, J. Wang, C.H. Yu, Y. Zhou, J. Cong, Z. Zhang, HeteroCL: A multi-paradigm programming infrastructure for software-defined reconfigurable computing, in: Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2019, pp. 242–251.

[122] Y. Ma, Y. Cao, S. Vrudhula, J.-s. Seo, An automatic RTL compiler for high-throughput FPGA implementation of diverse deep convolutional neural networks, in: 2017 27th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2017, pp. 1–8.

[123] Y. Guan, N. Xu, C. Zhang, Z. Yuan, J. Cong, Using data compression for optimizing FPGA-based convolutional neural network accelerators, in: International Workshop on Advanced Parallel Processing Technologies, Springer, 2017, pp. 14–26.

[124] Y. Ma, N. Suda, Y. Cao, J.-s. Seo, S. Vrudhula, Scalable and modularized RTL compilation of convolutional neural networks onto FPGA, in: 2016 26th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2016, pp. 1–8.

[125] S. Xu, B.C. Schafer, Approximating behavioral HW accelerators through selective partial extractions onto synthesizable predictive models, in: 2019 IEEE/ACM International Conference on Computer-Aided Design, ICCAD, IEEE, 2019, pp. 1–8.

[126] Y. You, Y. Chang, W. Wu, B. Guo, H. Luo, X. Liu, B. Liu, K. Zhao, S. He, L. Li, et al., New paradigm of FPGA-based computational intelligence from surveying the implementation of DNN accelerators, Des. Autom. Embedded Syst. (2022) 1–27.

[127] K. Zeng, Q. Ma, J.W. Wu, Z. Chen, T. Shen, C. Yan, FPGA-based accelerator for object detection: A comprehensive survey, J. Supercomput. 78 (12) (2022) 14096–14136.

[128] A. Saidi, S.B. Othman, M. Dhouibi, S.B. Saoud, FPGA-based implementation of classification techniques: A survey, Integration 81 (2021) 280–299.

[129] K. Guo, S. Zeng, J. Yu, Y. Wang, H. Yang, A Survey of FPGA-Based Neural Network Inference Accelerator (2018), arXiv preprint arXiv:1712.08934.

[130] R. Tapiador, A. Rios-Navarro, A. Linares-Barranco, M. Kim, D. Kadetotad, J.-s. Seo, Comprehensive evaluation of opencl-based convolutional neural network accelerators in xilinx and altera fpgas, 2016, arXiv preprint arXiv:1609.09296.

[131] U. Aydonat, S. O'Connell, D. Capalija, A.C. Ling, G.R. Chiu, An OpenCL™ deep learning accelerator on arria 10, in: Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2017, pp. 55–64.

[132] A. Lavin, S. Gray, Fast algorithms for convolutional neural networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 4013–4021.

[133] A. Aimar, H. Mostafa, E. Calabrese, A. Rios-Navarro, R. Tapiador-Morales, I.-A. Lungu, M.B. Milde, F. Corradi, A. Linares-Barranco, S.-C. Liu, et al., NullHop: A flexible convolutional neural network accelerator based on sparse representations of feature maps, IEEE Trans. Neural Netw. Learn. Syst. 30 (3) (2018) 644–656.

[134] R. Zhao, X. Niu, Y. Wu, W. Luk, Q. Liu, Optimizing CNN-based object detection algorithms on embedded FPGA platforms, in: Applied Reconfigurable Computing: 13th International Symposium, ARC 2017, Delft, the Netherlands, April 3-7, 2017, Proceedings 13, Springer, 2017, pp. 255–267.

[135] S. Arish, S. Sinha, K. Smitha, Optimization of convolutional neural networks on resource constrained devices, in: 2019 IEEE Computer Society Annual Symposium on VLSI, ISVLSI, IEEE, 2019, pp. 19–24.

[136] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O'Leary, R. Genov, A. Moshovos, Bit-pragmatic deep neural network computing, in: Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, 2017, pp. 382–394.

[137] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S.W. Keckler, W.J. Dally, SCNN: An accelerator for compressed-sparse convolutional neural networks, ACM SIGARCH Comput. Archit. News 45 (2) (2017) 27–40.

[138] T. Geng, A. Li, T. Wang, C. Wu, Y. Li, R. Shi, W. Wu, M. Herbordt, O3BNN-R: An out-of-order architecture for high-performance and regularized BNN inference, IEEE Trans. Parallel Distrib. Syst. 32 (1) (2020) 199–213.

[139] V. Akhlaghi, A. Yazdanbakhsh, K. Samadi, R.K. Gupta, H. Esmaeilzadeh, Snapea: Predictive early activation for reducing computation in deep convolutional neural networks, in: 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture, ISCA, IEEE, 2018, pp. 662–673.

[140] M. Song, J. Zhao, Y. Hu, J. Zhang, T. Li, Prediction based execution on deep neural networks, in: 2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture, ISCA, IEEE, 2018, pp. 752–763.

[141] S.W. Nabi, W. Vanderbauwhede, FPGA design space exploration for scientific HPC applications using a fast and accurate cost model based on roofline analysis, J. Parallel Distrib. Comput. 133 (2019) 407–419.

[142] M. Siracusa, L. Di Tucci, M. Rabozzi, S. Williams, E.D. Sozzo, M.D. Santambrogio, A CAD-based methodology to optimize HLS code via the roofline model, in: Proceedings of the 39th International Conference on Computer-Aided Design, 2020, pp. 1–9.

[143] S. Xu, S. Liu, Y. Liu, A. Mahapatra, M. Villaverde, F. Moreno, B.C. Schafer, Design space exploration of heterogeneous MPSoCs with variable number of hardware accelerators, Microprocess. Microsyst. 65 (2019) 169–179.

[144] M. Siracusa, E. Del Sozzo, M. Rabozzi, L. Di Tucci, S. Williams, D. Sciuto, M.D. Santambrogio, A comprehensive methodology to optimize FPGA designs via the roofline model, IEEE Trans. Comput. 71 (8) (2021) 1903–1915.

[145] G. Zhong, A. Prakash, S. Wang, Y. Liang, T. Mitra, S. Niar, Design space exploration of FPGA-based accelerators with multi-level parallelism, in: Design, Automation & Test in Europe Conference & Exhibition, DATE, 2017, IEEE, 2017, pp. 1141–1146.

[146] G. Zhong, A. Prakash, Y. Liang, T. Mitra, S. Niar, Lin-analyzer: A high-level performance analysis tool for FPGA-based accelerators, in: Proceedings of the 53rd Annual Design Automation Conference, 2016, pp. 1–6.

[147] L. Piccolboni, P. Mantovani, G.D. Guglielmo, L.P. Carloni, COSMOS: Coordination of high-level synthesis and memory optimization for hardware accelerators, ACM Trans. Embedd. Comput. Syst. (TECS) 16 (5s) (2017) 1–22.

[148] Q. Gautier, A. Althoff, C.L. Crutchfield, R. Kastner, Sherlock: A multi-objective design space exploration framework, ACM Trans. Des. Autom. Electron. Syst. (TODAES) 27 (4) (2022) 1–20.

[149] S. Dai, Y. Zhou, H. Zhang, E. Ustun, E.F. Young, Z. Zhang, Fast and accurate estimation of quality of results in high-level synthesis with machine learning, in: 2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines, FCCM, IEEE, 2018, pp. 129–132.

[150] L. Ferretti, J. Kwon, G. Ansaloni, G. Di Guglielmo, L.P. Carloni, L. Pozzi, Leveraging prior knowledge for effective design-space exploration in high-level synthesis, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 39 (11) (2020) 3736–3747.

[151] J. Kwon, L.P. Carloni, Transfer learning for design-space exploration with high-level synthesis, in: Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD, 2020, pp. 163–168.

[152] C. Lo, P. Chow, Model-based optimization of high level synthesis directives, in: 2016 26th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2016, pp. 1–10.

[153] L. Ferretti, G. Ansaloni, L. Pozzi, Cluster-based heuristic for high level synthesis design space exploration, IEEE Trans. Emerg. Top. Comput. 9 (1) (2018) 35–43.

[154] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, B. He, COMBA: A comprehensive model-based analysis framework for high level synthesis of real applications, in: 2017 IEEE/ACM International Conference on Computer-Aided Design, ICCAD, IEEE, 2017, pp. 430–437.

[155] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, B. He, Performance modeling and directives optimization for high-level synthesis on FPGA, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 39 (7) (2019) 1428–1441.

[156] K. Feng, X. Fan, J. An, C. Li, K. Di, J. Li, ACDSE: A design space exploration method for CNN accelerator based on adaptive compression mechanism, ACM Trans. Embedd. Comput. Syst. (TECS) (2022).

[157] A. Mehrabi, A. Manocha, B.C. Lee, D.J. Sorin, Bayesian optimization for efficient accelerator synthesis, ACM Trans. Archit. Code Optim. (TACO) 18 (1) (2020) 1–25.

[158] Y.-k. Choi, J. Cong, HLS-based optimization and design space exploration for applications with variable loop bounds, in: 2018 IEEE/ACM International Conference on Computer-Aided Design, ICCAD, IEEE, 2018, pp. 1–8.

[159] N. Wu, Y. Xie, C. Hao, Ironman: Gnn-assisted design space exploration in high-level synthesis via reinforcement learning, in: Proceedings of the 2021 on Great Lakes Symposium on VLSI, 2021, pp. 39–44.

[160] L. Ferretti, G. Ansaloni, L. Pozzi, Lattice-traversing design space exploration for high level synthesis, in: 2018 IEEE 36th International Conference on Computer Design, ICCD, IEEE, 2018, pp. 210–217.

[161] K. Feng, X. Fan, J. An, X. Wang, K. Di, J. Li, M. Lu, C. Li, Erdse: Efficient reinforcement learning based design space exploration method for CNN accelerator on resource limited platform, Graph. Visual Comput. 4 (2021) 200024.

[162] A.B. Perina, J. Becker, V. Bonato, Lina: Timing-constrained high-level synthesis performance estimator for fast DSE, in: 2019 International Conference on Field-Programmable Technology, ICFPT, IEEE, 2019, pp. 343–346.

[163] Y.-k. Choi, J. Cong, HLScope: High-level performance debugging for FPGA designs, in: 2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines, FCCM, IEEE, 2017, pp. 125–128.

[164] K. O'Neal, M. Liu, H. Tang, A. Kalantar, K. DeRenard, P. Brisk, HLSPredict: Cross platform performance prediction for FPGA high-level synthesis, in: Proceedings of the International Conference on Computer-Aided Design, 2018, pp. 1–8.

[165] B. Da Silva, A. Braeken, E.H. D'Hollander, A. Touhafi, Performance modeling for FPGAs: Extending the roofline model with high-level synthesis tools, Int. J. Reconfigur. Comput. 2013 (2013) 7.

[166] T. Nguyen, S. Williams, M. Siracusa, C. MacLean, D. Doerfler, N.J. Wright, The performance and energy efficiency potential of FPGAs in scientific computing, in: 2020 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems, PMBS, IEEE, 2020, pp. 8–19.

[167] H.M. Makrani, F. Farahmand, H. Sayadi, S. Bondi, S.M.P. Dinakarrao, H. Homayoun, S. Rafatirad, Pyramid: Machine learning framework to estimate the optimal timing and resource usage of a high-level synthesis design, in: 2019 29th International Conference on Field Programmable Logic and Applications, FPL, IEEE, 2019, pp. 397–403.

[168] Y. Liang, S. Wang, W. Zhang, FlexCL: A model of performance and power for OpenCL workloads on FPGAs, IEEE Trans. Comput. 67 (12) (2018) 1750–1764.

[169] E. Calore, S.F. Schifano, Performance assessment of FPGAs as HPC accelerators using the FPGA empirical roofline, in: 2021 31st International Conference on Field-Programmable Logic and Applications, FPL, IEEE, 2021, pp. 83–90.

[170] K. Papadimitriou, A. Dollas, S. Hauck, Performance of partial reconfiguration in FPGA systems: A survey and a cost model, ACM Trans. Reconfigur. Technol. Syst. (TRETS) 4 (4) (2011) 1–24.

[171] M. Hora, V. Končický, J. Těték, Theoretical model of computation and algorithms for FPGA-based hardware accelerators, in: Theory and Applications of Models of Computation: 15th Annual Conference, TAMC 2019, Kitakyushu, Japan, April 13–16, 2019, Proceedings 15, Springer, 2019, pp. 295–312.

[172] Z. Lin, Z. Yuan, J. Zhao, W. Zhang, H. Wang, Y. Tian, Powergear: Early-stage power estimation in FPGA HLS via heterogeneous edge-centric GNNs, in: 2022 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE, 2022, pp. 1341–1346.

[173] Z. Wang, B. He, W. Zhang, S. Jiang, A performance analysis framework for optimizing OpenCL applications on FPGAs, in: 2016 IEEE International Symposium on High Performance Computer Architecture, HPCA, IEEE, 2016, pp. 114–125.

[174] N. Kapre, H. Patel, Applying models of computation to OpenCL pipes for FPGA computing, in: Proceedings of the 5th International Workshop on OpenCL, 2017, pp. 1–4.
[175] Y.S. Shao, B. Reagen, G.-Y. Wei, D. Brooks, Aladdin: A PRE-RTL, power-performance accelerator simulator enabling large design space exploration of customized architectures, ACM SIGARCH Comput. Archit. News 42 (3) (2014) 97–108.
[176] M. Makni, S. Niar, M. Baklouti, M. Abid, HAPE: A high-level area-power estimation framework for FPGA-based accelerators, Microprocess. Microsyst. 63 (2018) 11–27.
[177] J.J. Davis, E. Hung, J.M. Levine, E.A. Stott, P.Y. Cheung, G.A. Constantinides, KAPow: High-accuracy, low-overhead online per-module power estimation for FPGA designs, ACM Trans. Reconfigur. Technol. Syst. (TRETS) 11 (1) (2018) 1–22.
[178] J. Lorandel, J.-C. Prévotet, M. Hélard, Efficient modelling of FPGA-based IP blocks using neural networks, in: 2016 International Symposium on Wireless Communication Systems, ISWCS, IEEE, 2016, pp. 571–575.
[179] Y. Nasser, J.-C. Prévotet, M. Hélard, Power modeling on FPGA: A neural model for RT-level power estimation, in: Proceedings of the 15th ACM International Conference on Computing Frontiers, 2018, pp. 309–313.
[180] A.N. Tripathi, A. Rajawat, An accurate and quick ANN-based system-level dynamic power estimation model using LLVM IR profiling for FPGA designs, IEEE Embedd. Syst. Lett. 12 (2) (2019) 58–61.
[181] G. Verma, V. Khare, M. Kumar, More precise FPGA power estimation and validation tool (FPEV_Tool) for low power applications, Wirel. Pers. Commun. 106 (2019) 2237–2246.
[182] Z. Lin, J. Zhao, S. Sinha, W. Zhang, HL-Pow: A learning-based power modeling framework for high-level synthesis, in: 2020 25th Asia and South Pacific Design Automation Conference, ASP-DAC, IEEE, 2020, pp. 574–580.
[183] G. Verma, T. Singhal, R. Kumar, S. Chauhan, S. Shekhar, B. Pandey, D. Akbar Hussain, Heuristic and statistical power estimation model for FPGA based wireless systems, Wirel. Pers. Commun. 106 (2019) 2087–2098.
[184] F. Farahmand, A. Ferozpuri, W. Diehl, K. Gaj, Minerva: Automated hardware optimization tool, in: 2017 International Conference on ReConFigurable Computing and FPGAs, ReConFig, IEEE, 2017, pp. 1–8.
[185] V. Manohararajah, S.D. Brown, Z.G. Vranesic, Heuristics for area minimization in LUT-based FPGA technology mapping, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 25 (11) (2006) 2331–2340.